

Entwicklungstools: GitHub im agilen Projektmanagement

Inhaltsverzeichnis

- 7 GitHub im agilen Projekt — Setup und Workflow mit Scrumban
 - 7.1 Projektstart und Setup
 - 7.1.1 Repository erstellen und konfigurieren
 - 7.1.2 Grundlegende Dateien
 - 7.1.3 Collaborators und Zugriffsverwaltung
 - 7.1.4 Branch Protection Rules einrichten
 - 7.2 GitHub-Planungswerkzeuge
 - 7.2.1 Was ist das GitHub Project Board?
 - 7.2.2 Roadmap-Ansicht für Zeitplanung und Überblick
 - 7.2.3 GitHub Issues als zentrale Aufgabenverwaltung
 - 7.2.4 Issue-Templates definieren und konfigurieren
 - 7.2.5 Milestones und Release-Planung
 - 7.3 Entwicklungsworkflow mit GitHub Flow
 - 7.3.1 Branching-Strategien im Vergleich
 - 7.3.2 Branch-Naming-Konventionen
 - 7.3.3 Pull Requests und Code Reviews
 - 7.3.4 PR-Templates definieren
 - 7.3.5 Verbindung zwischen Issues, Branches und Pull Requests
 - 7.4 CI/CD mit GitHub Actions
 - 7.4.1 Anatomie eines GitHub Actions Workflows
 - 7.4.2 CI-Workflow bei Pull Requests
 - 7.4.3 GitHub-hosted Runner vs. Self-hosted Runner
 - 7.4.4 Self-hosted Runner lokal einrichten
 - 7.4.5 Test-Instanzen und Deployment-Previews

7 GitHub im agilen Projekt — Setup und Workflow mit Scrumban

In modernen Softwareentwicklungsprojekten wie unserem Schulbibliotheks-System stehen Agilität, kontinuierliche Integration und kollaboratives Arbeiten im Mittelpunkt. GitHub bietet mit seinen Funktionen wie Repositories, Branching, Issues, Pull Requests, Projects (Boards) und Automatisierungen ein integriertes Ökosystem, das agile Methoden wirkungsvoll unterstützen kann. Dieses Kapitel zeigt, wie GitHub bei einem agilen oder hybriden Vorgehen wie Scrumban eingerichtet und praktisch eingesetzt werden kann — von Projektstart über den täglichen Workflow bis zur kontinuierlichen Lieferung und Qualitätssicherung.



Merksatz: GitHub ist mehr als nur ein Versionskontrollsystem – es ist eine integrierte Plattform für agile Zusammenarbeit, die Issues, Boards, Code Reviews, Automatisierung und Dokumentation unter einem Dach vereint.

7.1 Projektstart und Setup

Bevor wir mit der Entwicklung des Schulbibliotheks-Systems beginnen, müssen wir die Grundlagen für eine effiziente Zusammenarbeit schaffen. Der Projektstart auf GitHub umfasst mehrere wichtige Schritte, die wir im Folgenden detailliert betrachten.

Stellen Sie sich das GitHub-Repository wie ein gemeinsames Büro vor: Bevor das Team einzieht und loslegt, braucht es eine klare Raumaufteilung (Ordnerstruktur), Hausregeln (CONTRIBUTING.md), ein Türschild (README.md) und eine Zugangskontrolle (Branch Protection Rules). Erst dann arbeitet jeder produktiv und ohne Chaos.

Lernziele dieses Abschnitts:

- Ein neues GitHub-Repository sinnvoll strukturieren und konfigurieren
- Die wichtigsten Basis-Dateien (README, .gitignore, CONTRIBUTING) kennen und erstellen
- Branch Protection Rules verstehen und für das Schulprojekt einrichten

7.1.1 Repository erstellen und konfigurieren

Zu Projektbeginn legen wir auf GitHub ein neues Repository für unser Projekt (als Beispiel hier Schulbibliotheks-System) an. Die Wahl einer durchdachten Ordnerstruktur ist entscheidend für die langfristige Wartbarkeit des Projekts.

Git bzw. GitHub basiert auf **verteilten Versionskontrollsystemen (DVCS)**. Jeder Entwickler hat eine vollständige Kopie der Projekthistorie lokal auf seinem Rechner — das gewährleistet Unabhängigkeit, Geschwindigkeit und automatisches Backup.

Repository auf GitHub anlegen

1. Melde dich auf github.com an

2. Klicke auf **"New"** (grüner Button oben links) oder **"+"** → **"New repository"**

3. Fülle das Formular aus:

- **Repository name:** `schulbibliothek` (kleinbuchstaben, Bindestriche statt Leerzeichen)
- **Description:** Kurze Projektbeschreibung
- **Visibility:** `Private` (für Schulprojekte empfohlen) oder `Public`
- ☒ **Add a README file** — wichtige Basisdatei, direkt miterstellen
- ☒ **Add .gitignore** → Template `Maven` auswählen (Für unterschiedliche Projektarten kann man sich diese über die Seite [Topal - .gitignore](#) erstellen lassen)
- **License:** Optional, für Schulprojekte meist nicht notwendig
- **Project Prompt** (optional): Mit einem Prompt kann das Projekt beschrieben werden (z.B. Angabe des Technologie Stacks, grundlegende Ordnerstruktur, Thema) wodurch mittels Github Copilot ein Projekt, basierend auf der Beschreibung, in einem eigenen Branch angelegt wird.

4. Klicke auf **"Create repository"**

Beispiel Ordnerstruktur für die Digitale Schulbibliothek (Clean Architecture)

Die Ordnerstruktur sollte die gewählte Software-Architektur widerspiegeln. Für unser Projektbeispiel mit **Clean Architecture** und dem Maven-Build-System wählen wir:

```
schulbibliothek/
├── docs/                # Dokumentation
│   ├── architecture/   # Architekturdiagramme und -beschreibungen
│   └── api/            # API-Spezifikationen (OpenAPI/Swagger),
└── AsyncAPI
    ├── user-stories/    # User Stories und Anforderungen
    └── src/
        ├── main/
        │   ├── java/
        │   │   └── com/schule/bibliothek/
        │   │       ├── domain/        # Core Domain (Ausleih-Kontext)
        │   │       │   ├── entities/  # Ausleiher, AusleihExemplar
        │   │       │   ├── valueobjects/ # ISBN, Signatur, Rückgabedatum
        │   │       │   └── services/   # MahnService
        │   │       ├── application/    # Application Services / Use Cases
        │   │       ├── infrastructure/ # Repositories, Datenbank, JPA
        │   │       └── presentation/   # REST Controller, DTOs
        │   └── resources/
        │       ├── application.yml      # Spring Boot Konfiguration
        │       └── db/migration/        # Flyway/Liquibase Migrations
        └── test/
            ├── java/
            │   ├── com/schule/bibliothek/
            │   │   ├── unit/           # Unit Tests
            │   │   ├── integration/    # Integration Tests
            │   │   └── e2e/            # End-to-End Tests
            └── resources/
                └── application-test.yml
└── pom.xml                # Maven Build-Konfiguration
```

```

├── README.md           # Projektbeschreibung (automatisch angelegt)
├── .gitignore          # Ignorierte Dateien (automatisch angelegt)
├── CONTRIBUTING.md     # Beitragsrichtlinien für das Team
├── .github/
│   ├── workflows/     # GitHub Actions für CI/CD
│   ├── ISSUE_TEMPLATE/ # Issue-Templates
│   └── pull_request_template.md # PR-Template

```



Merksatz: Bei **Clean Architecture** trennen wir klar zwischen **domain**, **application**, **infrastructure** und **presentation**. Diese Trennung macht die Bounded Contexts aus dem DDD-Modell (siehe Kapitel 4.1) direkt im Code sichtbar. Die Standard-Maven-Struktur mit **src/main/java** und **src/test/java** wird dabei beibehalten.



Vertiefung: Was ist DVCS?

Bei einem **Distributed Version Control System** wie Git hat jeder Entwickler eine vollständige Kopie des Repositories inklusive aller Commits und der gesamten Historie. Das unterscheidet sich von zentralisierten Systemen (z.B. SVN), bei denen nur ein Server die vollständige Historie hält.

- **Offline-Arbeit:** Commits, Branches und Diffs funktionieren ohne Internetverbindung
- **Backup:** Jeder Klon ist ein vollständiges Backup
- **Performance:** Lokale Operationen sind sehr schnell
- **Parallele Workflows:** Mehrere Teams können unabhängig voneinander arbeiten

7.1.2 Grundlegende Dateien

Neben der Ordnerstruktur gibt es drei Dateien, die in jedem professionellen Repository vorhanden sein sollten: **README.md**, **.gitignore** und **CONTRIBUTING.md**. Sie bilden die Grundlage für eine effiziente Teamzusammenarbeit.

README.md — Das Türschild des Projekts

Die **README.md** ist die erste Datei, die Besucher des Repositories sehen. Sie wird auf der Startseite des Repositories automatisch gerendert und sollte alle wichtigen Informationen auf einen Blick liefern.

Empfohlene Struktur für das Beispielprojekt Schulbibliothek:

```
# Schulbibliotheks-System
```

```
> Digitales Verwaltungssystem für die Schulbibliothek – entwickelt als SYP-Projekt
```

```
## Beschreibung
```

```
Kurze Beschreibung des Projekts: Was wird entwickelt? Für wen? Welches Problem wird gelöst?
```

```
## Technologie-Stack
```

- **Backend:** Java 17, Spring Boot 3.x
- **Datenbank:** PostgreSQL
- **Frontend:** (z.B. React / Flutter)
- **Build:** Maven
- **CI/CD:** GitHub Actions

Voraussetzungen

- Java 17 (JDK)
- Maven 3.9+
- Docker (für lokale Datenbank)

Schnellstart

```

```bash
Repository klonen
git clone https://github.com/schule/schulbibliothek.git
cd schulbibliothek

Datenbank starten (Docker)
docker compose up -d postgres

Anwendung starten
mvn spring-boot:run

Projektstruktur

Kurze Erklärung der wichtigsten Verzeichnisse (siehe oben)

Team

| Name | Rolle | GitHub |
|-----|-----|-----|
| Max Muster | Backend | @maxmuster |

Lizenz

Schulprojekt – alle Rechte vorbehalten

```



**Praxis-Tipp:** Eine gute README spart dem Team Zeit. Neue Mitglieder können das Projekt eigenständig aufsetzen, ohne jemanden fragen zu müssen. Halte die README immer aktuell — veraltete Installationsanleitungen sind schlimmer als gar keine.

## .gitignore — Was Git ignorieren soll

Die `.gitignore`-Datei legt fest, welche Dateien und Verzeichnisse Git **nicht** verwalten soll. Ohne sie würden z.B. kompilierte Klassen, IDE-Konfigurationen oder Passwörter versehentlich ins Repository gelangen.

## Wichtige Kategorien für ein Java/Spring-Projekt:

```
===== Build-Ausgaben =====
target/
*.class
*.jar
*.war

===== IDE-spezifisch =====
.idea/ # IntelliJ IDEA
*.iml
.vscode/ # VS Code (außer geteilte Settings)
.eclipse/

===== Betriebssystem =====
.DS_Store # macOS
Thumbs.db # Windows
desktop.ini

===== Konfiguration mit Secrets =====
application-local.yml # Lokale Konfiguration mit Passwörtern
*.env # Umgebungsvariablen
secrets/

===== Logs =====
*.log
logs/

===== Test-Ausgaben =====
/test-results/
/coverage/
```



**Achtung:** Secrets (Passwörter, API-Keys, Zugangsdaten) dürfen **niemals** in ein Repository gelangen — auch nicht in ein `private`! Nutze stattdessen Umgebungsvariablen oder GitHub Secrets für CI/CD-Pipelines. Einmal commitete Secrets müssen aus der gesamten Git-Historie entfernt werden, was aufwändig ist.



**Merksatz:** GitHub bietet unter [github.com/github/gitignore](https://github.com/github/gitignore) vorgefertigte `.gitignore`-Templates für alle gängigen Sprachen und Frameworks. Beim Erstellen des Repositories kann direkt ein Template ausgewählt werden.

## CONTRIBUTING.md — Die Hausregeln des Teams

Die `CONTRIBUTING.md` beschreibt, wie das Team zusammenarbeitet. Sie ist besonders wichtig für neue Mitglieder und stellt sicher, dass alle nach denselben Regeln arbeiten.

### Empfohlene Inhalte für die Schulbibliothek:

# Beitragsrichtlinien (Contributing Guidelines)

## ## Entwicklungsworkflow

1. **\*\*Issue auswählen\*\*** aus dem Backlog (GitHub Project Board)
2. **\*\*Branch erstellen\*\*** nach Namenskonvention (siehe unten)
3. **\*\*Entwicklung\*\*** mit regelmäßigen, aussagekräftigen Commits
4. **\*\*Pull Request\*\*** öffnen mit ausgefülltem PR-Template
5. **\*\*Code Review\*\*** durch mindestens 1 Teammitglied
6. **\*\*Merge\*\*** nach Approval und grünen CI-Checks

## ## Branch-Namenskonvention

feature/[issue-nr]-[kurzbeschreibung]	→ feature/42-mahnwesen-automation
bugfix/[issue-nr]-[kurzbeschreibung]	→ bugfix/57-rueckgabedatum-fehler
hotfix/[kurzbeschreibung]	→ hotfix/login-absturz
doc/[kurzbeschreibung]	→ doc/api-dokumentation-aktualisieren
refactor/[kurzbeschreibung]	→ refactor/domain-services-optimieren

## ## Commit-Konvention (Conventional Commits)

Format: `<type>(<scope>): <beschreibung>`

Typen: ``feat``, ``fix``, ``docs``, ``test``, ``refactor``, ``chore``

Scopes: ``domain``, ``application``, ``infrastructure``, ``presentation``

Beispiele:

- ``feat(domain): Add Mahnung entity to Ausleiher aggregate``
- ``fix(infrastructure): Correct connection pool configuration``
- ``test(domain): Add unit tests for AusleihLimit validation``

## ## Code-Qualitätsregeln

- Mindestens 80% Test-Abdeckung im Domain-Layer
- Clean Architecture Schichtentrennung einhalten
- Kein direkter Push auf ``main`` – immer via Pull Request
- Alle CI-Checks müssen grün sein vor dem Merge



### Vertiefung: Conventional Commits

**Conventional Commits** ist ein Standard für Commit-Nachrichten. Durch die strukturierte Form `<type>(<scope>): <beschreibung>` können Tools automatisch Changelogs generieren und Semantic Versioning bestimmen:

- `feat` → neue Funktion → Minor-Version-Bump (1.0.0 → 1.1.0)
- `fix` → Bugfix → Patch-Version-Bump (1.0.0 → 1.0.1)
- `feat!` oder `BREAKING CHANGE` → Major-Version-Bump (1.0.0 → 2.0.0)

Mehr unter: [conventionalcommits.org](https://conventionalcommits.org)

## Quellen

- [About READMEs - GitHub Docs](#)

- [gitignore.io](https://gitignore.io) — Generator für .gitignore-Dateien
- [Conventional Commits Specification](https://conventionalcommits.org/)

---

### 7.1.3 Collaborators und Zugriffsverwaltung

In einem Schulprojekt ist klare Zugriffsverwaltung genauso wichtig wie eine gute Ordnerstruktur. GitHub erlaubt es, Projektbeteiligte als **Collaborators** mit unterschiedlichen Berechtigungen einzuladen und so den Zugang gezielt zu steuern.

#### Best Practices für Collaborators

- **Least Privilege:** Vergib nur die Rechte, die der Nutzer wirklich braucht. Entwickler erhalten meist **Write**, Projektleitung oder Lehrer **Maintain** bzw. **Admin**.
- **Keine gemeinsamen Accounts:** Jeder arbeitet mit seinem eigenen GitHub-Konto. Das erhöht Transparenz und Nachvollziehbarkeit.
- **Regelmäßige Überprüfung:** Entferne Zugänge, wenn Teammitglieder das Projekt verlassen oder die Rolle sich ändert.
- **2FA für Organisationen:** Wenn das Projekt in einer Organisation liegt, sollte Zwei-Faktor-Authentifizierung (2FA) Pflicht sein.
- **Teams statt Einzelpersonen:** Bei GitHub-Organisationen ist es besser, Nutzer in Teams zu organisieren, statt sie einzeln als Collaborators hinzuzufügen.
- **Verwende Teams mit definierten Rollen:** **Owners**, **Maintainers**, **Contributors** helfen bei der Skalierung.

#### Rollen und Empfehlungen

Rolle	Beschreibung	Empfehlung für das Schulprojekt
<b>Read</b>	Nur Lesezugriff auf Code und Issues	Für Beobachter / Stakeholder, wenn benötigt
<b>Triage</b>	Issues und PRs verwalten, aber keinen Code pushen	Für Scrum Master / Projektleiter ohne Coding-Aufgaben
<b>Write</b>	Code pushen, Issues und PRs bearbeiten	Für die meisten Entwickler im Team
<b>Maintain</b>	Repository-Einstellungen ändern, Branches verwalten	Für Teamleitung, Scrum Master oder Lehrer
<b>Admin</b>	Voller Zugriff, inkl. Sicherheits- und Berechtigungseinstellungen	Nur für Projektverantwortliche und Lehrkräfte

#### Collaborators einladen

1. Öffne dein Repository und gehe zu **Settings** → **Collaborators and teams**
2. Klicke auf **Add people** oder **Add teams**
3. Suche nach dem GitHub-Benutzernamen oder Team
4. Wähle die passende Berechtigung aus (**Read**, **Triage**, **Write**, **Maintain**, **Admin**)

## 5. Schicke die Einladung ab



**Merksatz:** Ein GitHub-Repository sollte mindestens einen Admin haben, aber nicht alle Teammitglieder brauchen Admin-Rechte. Der Standardfall im Schulprojekt ist: Entwickler = **Write**, Teamleitung = **Maintain**, Lehrkraft = **Admin**.

## Zugriffsverwaltung im Schulprojekt

- **Vermeide zu viele Admins:** Admin-Zugriff sollte auf wenige verantwortliche Personen beschränkt werden.
- **Nutze Branch Protection zusammen mit Collaborators:** Dadurch können Entwickler zwar pushen, aber nicht direkt auf **main** mergen.
- **Audit-Log prüfen:** Bei Problemen zeigt das GitHub-Audit-Log, wer wann welche Einstellungen geändert hat.

## Alternative: Organisation + Teams

Wenn das Projekt in einer GitHub-Organisation liegt, ist das sicherer als einzelne Repository-Collaborators:

- Erstelle ein Team wie **schulbibliothek-devs**
- Füge alle Entwickler diesem Team hinzu
- Vergib dem Team die Rolle **Write**
- Nutze ein separates Team **schulbibliothek-admins** für **Maintain** oder **Admin**



**Achtung:** Jeder, der **Admin**-Rechte hat, kann Branch Protection Rules ändern oder Collaborators verwalten. Diese Rolle sollte nur wenigen Personen vertrauenswürdig übertragen werden.

## Quellen

- [Managing repository collaborators - GitHub Docs](#)
- [Repository permission levels for an organization - GitHub Docs](#)

## 7.1.4 Branch Protection Rules einrichten

Branch Protection Rules sind Sicherheitsmechanismen in GitHub, die verhindern, dass Code ohne Qualitätsprüfung auf wichtige Branches (z.B. **main**) gelangt. Sie erzwingen Prozesse wie Code Reviews und automatische Tests.

Stellen Sie sich Branch Protection Rules wie eine Qualitätskontrolle am Fabrikstor vor: Nur fertige, geprüfte Produkte dürfen das Gebäude verlassen. Halbfertige Arbeit bleibt in der Produktion (Feature-Branch), bis sie die Prüfstation (CI-Tests + Code Review) erfolgreich passiert hat.



**Achtung:** Ohne Branch Protection Rules sind alle CI-Status-Checks nur informativ — ein Merge ist auch bei roten Tests möglich. Erst mit aktivierten Rules wird die Qualitätssicherung wirklich erzwungen.

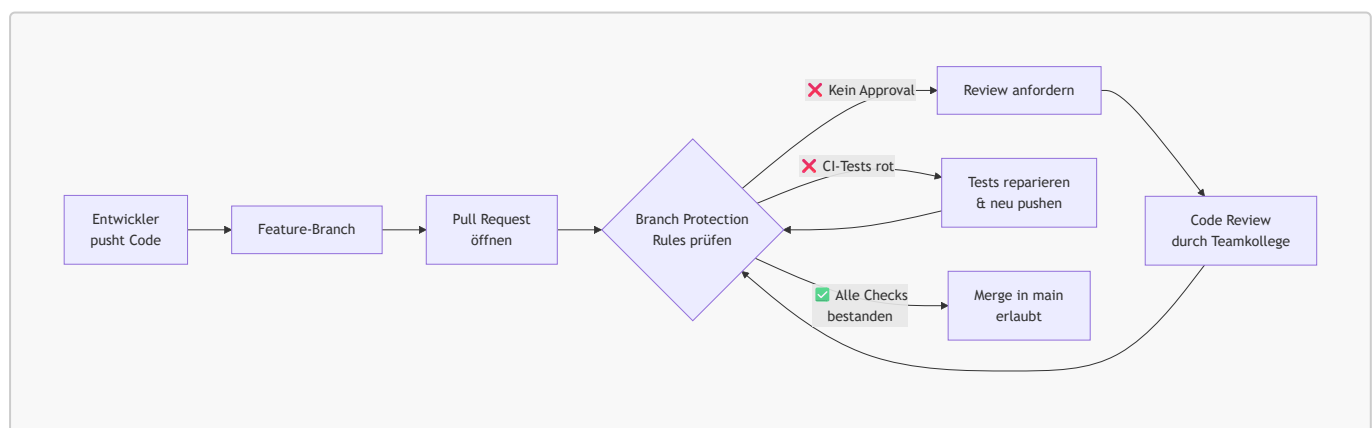
## Konfiguration in GitHub

1. Navigiere zu deinem Repository → **Settings** → **Rules** → **Rulesets**
2. Klicke auf **"New ruleset"** → **"New branch ruleset"**
3. Vergib einen Namen, z.B. **Protect main**
4. Unter **"Target branches"** → **"Add target"** → **"Include by pattern"** → **main** eingeben
5. Aktiviere die gewünschten Regeln (siehe Tabelle unten)
6. Klicke auf **"Create"**

### Empfohlene Regeln für die Schulbibliothek

Regel	Beschreibung	Empfehlung Schulprojekt
<b>Require a pull request before merging</b>	Kein direkter Push auf <b>main</b> erlaubt	✓ Aktivieren
<b>Required approvals</b>	Anzahl der erforderlichen Review-Approvals	✓ Mindestens 1
<b>Dismiss stale pull request approvals</b>	Approval verfällt bei neuem Push auf den Branch	✓ Aktivieren
<b>Require status checks to pass</b>	CI-Checks müssen grün sein	✓ Aktivieren (nach CI-Setup in Kap. 7.4)
<b>Require branches to be up to date</b>	Branch muss aktuell mit <b>main</b> sein vor dem Merge	✓ Aktivieren
<b>Restrict who can push to matching branches</b>	Nur bestimmte Personen dürfen direkt pushen	Optional (z.B. nur Projektleiter)
<b>Block force pushes</b>	<b>git push --force</b> wird verhindert	✓ Aktivieren

### Beispiel: Konfiguration für das Schulbibliotheks-Projekt



**Merksatz:** Branch Protection Rules sind kein Misstrauen gegenüber dem Team, sondern ein Sicherheitsnetz. Sie verhindern unbeabsichtigte Fehler, fördern Code Reviews als Lerngelegenheit und stellen sicher, dass **main** immer in einem lauffähigen Zustand ist.



**Praxis-Tipp:** Im Schulprojekt kann es sinnvoll sein, dass der Lehrer / Projektbetreuer als **Repository-Owner** die Möglichkeit behält, die Rules im Notfall zu umgehen (Option "Allow

*administrators to bypass configured branch protections*" deaktiviert lassen — dann gelten die Rules auch für Admins).

---

## Quellen

- [About protected branches - GitHub Docs](#)
  - [Managing rulesets for a repository - GitHub Docs](#)
-

## 7.2 GitHub-Planungswerkzeuge

In diesem Abschnitt lernen wir die zentralen GitHub-Planungswerkzeuge kennen: **Projects** für die visuelle Planung mit Scrumban-Boards, **Issues** als ausführliche Aufgaben- und Ticketverwaltung, **Issue-Templates** für konsistente Eingaben und **Milestones** zur Gruppierung und Zielverfolgung größerer Etappen.

Stellen Sie sich GitHub Projects wie eine digitale Pinnwand vor, auf der jede Aufgabe (Issue) als Haftnotiz klebt. Die Spalten zeigen den Fortschritt — von „Backlog“ über „In Progress“ bis „Done“. Milestones sind dabei die übergeordneten Projektziele, auf die alle Karten gemeinsam hinarbeiten.

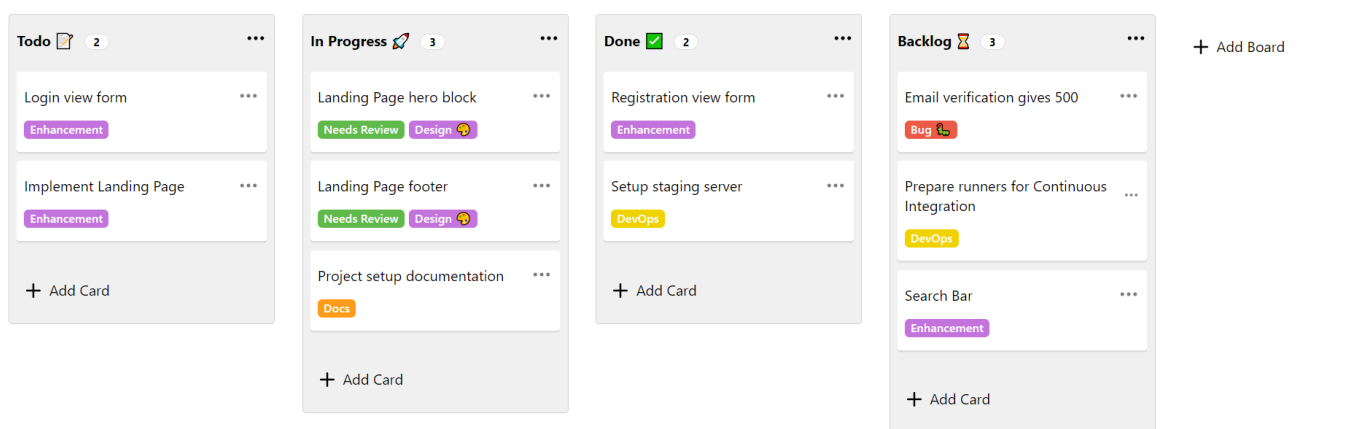
### Lernziele dieses Abschnitts:

- Ein GitHub Project Board für Scrumban einrichten und anpassen
- Issues sinnvoll strukturieren und Labels/Milestones zuweisen
- Issue-Templates für konsistente Eingaben im Team erstellen
- Milestones für Release-Planung einsetzen

### 7.2.1 Was ist das GitHub Project Board?

Das GitHub Project Board ist eine visuelle Arbeitsfläche innerhalb von GitHub, die Karten für Issues, Pull Requests und Notizen in Spalten organisiert und so den Scrumban-Workflow transparent macht.

#### Kanban Board



Das GitHub Project Board zeigt den Arbeitsfluss in Spalten — von Backlog bis Done.

Ein agiles Board besteht aus drei zentralen Elementen:

- **Spalten (Status-Felder):** Zeigen den aktuellen Fortschritt jeder Aufgabe — z.B. **Backlog**, **To Do**, **In Progress**, **Code Review**, **Testing**, **Done**
- **Cards:** Jede Karte repräsentiert ein Issue oder einen Pull Request
- **Custom Fields:** Zusatzinformationen pro Karte wie Priorität, Story Points oder Bounded Context

### Einrichtung des GitHub Project Boards

Die folgenden Schritte zeigen, wie Sie ein GitHub Project Board als zentrale Planungs- und Kollaborationsfläche für das Schulprojekt aufbauen.

## 1. Projekt erstellen

- Navigieren Sie in Ihrem Repository zum Reiter **"Projects"**
- Klicken Sie auf **"New project"** (oder "Link a project" → "New project")
- Wählen Sie das Template **"Board"** aus
- Geben Sie dem Projekt einen Namen, z.B. **Schulbibliothek Entwicklung**

## 2. Spalten (Status) konfigurieren

Standardmäßig erhalten Sie "Todo", "In Progress" und "Done". Wir passen diese an unseren Scrumban-Workflow an:

- Gehen Sie in die **Settings** (Zahnrad-Icon) → **Fields** → **Status**
- Ergänzen/bearbeiten Sie die Optionen:

Spalte	Farbe	Bedeutung
<b>Backlog</b>	Grau	Aufgaben, die noch nicht für den Sprint geplant sind
<b>To Do</b>	Rot	Im aktuellen Sprint geplant, noch nicht begonnen
<b>In Progress</b>	Gelb	Aktiv in Entwicklung
<b>Code Review</b>	Blau	Pull Request offen, wartet auf Review
<b>Testing</b>	Lila	Feature fertig, wird getestet
<b>Done</b>	Grün	Abgeschlossen und gemerkt

## 3. Custom Fields hinzufügen

Klicken Sie in der Tabellenansicht auf das "+" (New Field) und erstellen Sie folgende Felder:

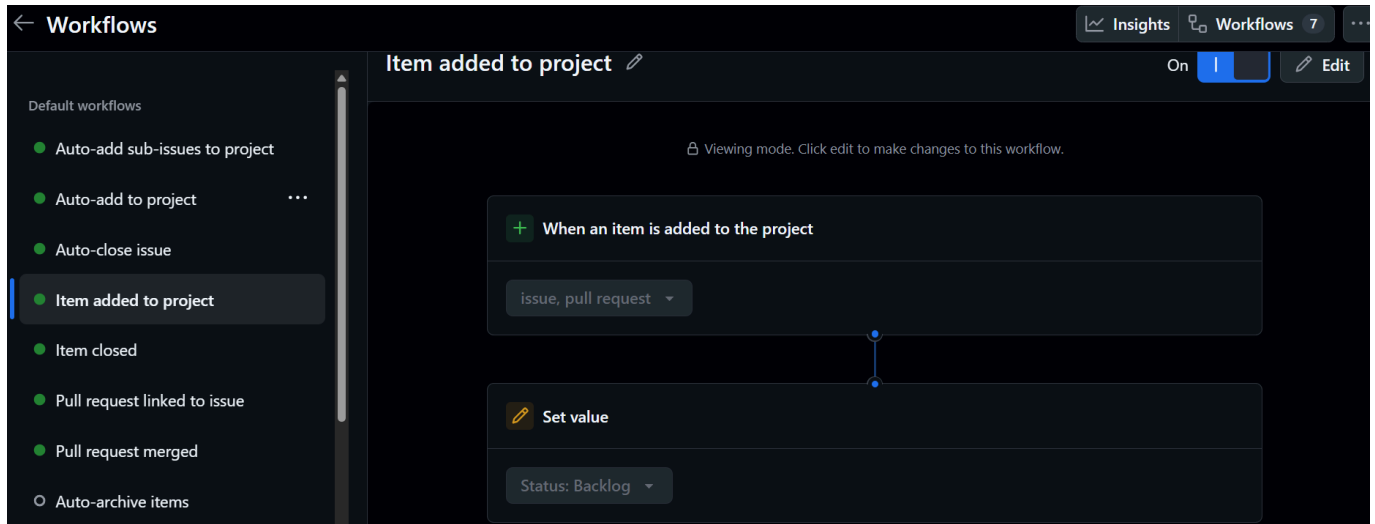
Feldname	Typ	Optionen
<b>Priority</b>	Single select	<b>High</b>  , <b>Medium</b>  , <b>Low</b> 
<b>Story Points</b>	Single select	<b>1, 2, 3, 5, 8, 13</b> (Fibonacci)
<b>Bounded Context</b>	Single select	<b>Ausleih, Anschaffung, Nutzerprofil</b>



**Merksatz:** Das GitHub Project Board verbindet Aufgabenplanung (Issues), Fortschrittsverfolgung (Spalten) und Kontextinformation (Custom Fields) in einer Ansicht — kein externes Tool notwendig.

## Automatisierung der Workflows für das Project Board

GitHub Projects bietet eingebaute **Standard-Workflows**, die Karten automatisch verschieben oder Statuswerte aktualisieren, sobald bestimmte Ereignisse eintreten — ohne manuellen Aufwand für das Team.



Links: aktivierbare Default-Workflows. Rechts: Konfiguration des ausgewählten Workflows — neue Items werden automatisch in die Spalte „Backlog“ verschoben.

### Aktivierte Automationen beim Projektstart:

Ereignis	Automatische Aktion
Issue geschlossen	Status → Done
Pull Request gemergt	Status → Done

### Weitere konfigurierbare Automationen:

- Neues Item dem Projekt hinzugefügt → Status → Backlog
- Pull Request geöffnet → Status → Code Review
- Issue wieder geöffnet → Status → To Do
- Item archivieren, wenn es Done erreicht

### Automation konfigurieren:

1. Öffnen Sie das Project Board
2. Klicken Sie auf das "..." Menü (oben rechts) → **"Workflows"**
3. Wählen Sie einen Workflow aus und klicken Sie auf **"Edit"**
4. Aktivieren oder deaktivieren Sie den Workflow mit dem Toggle

**Praxis-Tipp:** Aktivieren Sie zumindest den Workflow "Item added to project → set status to Backlog". Dadurch landen neue Issues automatisch im Backlog und das Board bleibt ohne manuellen Aufwand aktuell.

### Quellen

- [Using the built-in automations - GitHub Docs](#)

## 7.2.2 Roadmap-Ansicht für Zeitplanung und Überblick

Die **Roadmap-Ansicht** in GitHub Projects zeigt Issues, Pull Requests und Drafts auf einer Zeitachse. Sie ist ideal, um den zeitlichen Verlauf eines Projekts zu visualisieren, Phasen zu planen und den Fortschritt über Sprints und Milestones hinweg im Blick zu behalten.

Stellen Sie sich die Roadmap wie einen Projektkalender vor: Jedes Issue ist ein Balken auf der Zeitlinie — von Start- bis Zieldatum. So erkennt man auf einen Blick, welche Aufgaben sich überschneiden, wo Engpässe drohen und wann welche Meilensteine anstehen.

### Was leistet die Roadmap-Ansicht?

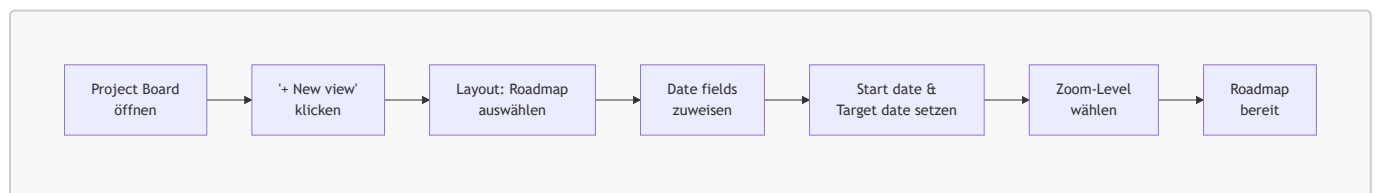
- **Zeitliche Visualisierung:** Issues werden als Balken auf einer Zeitlinie dargestellt — von **Start date** bis **Target date**
- **Zoom-Level:** Die Ansicht kann auf **Monat**, **Quartal** oder **Jahr** umgestellt werden — je nach benötigtem Detailgrad
- **Vertikale Marker:** Wichtige Datum-Referenzpunkte wie Milestones, Iterationen oder feste Termine können als vertikale Linien eingeblendet werden
- **Drag & Drop:** Start- und Enddaten können direkt in der Roadmap durch Ziehen verändert werden
- **Gruppierung:** Issues lassen sich nach Feldern gruppieren (z.B. nach Bounded Context, Sprint oder Priority)
- **Slicing:** Eine Seitenleiste filtert die Ansicht nach einem bestimmten Feldwert (z.B. nur **ausleih-kontext**)

### Roadmap-Ansicht einrichten

**Voraussetzung:** Im Projekt müssen die Felder **Start date** und **Target date** als Datumsfelder angelegt sein (Custom Fields — siehe Kapitel 7.2.1).

#### Neue Ansicht als Roadmap anlegen:

1. Im Project Board auf das "+" (neue Ansicht) klicken
2. Als Layout "**Roadmap**" auswählen
3. Über "**Date fields**" (oben rechts) die Felder **Start date** und **Target date** zuweisen
4. Zoom-Level je nach Planung auf **Month**, **Quarter** oder **Year** setzen



### Vertikale Marker konfigurieren

Marker helfen, wichtige Zeitpunkte sichtbar zu machen:

1. Oben rechts auf "**Markers**" klicken
2. Gewünschte Marker aktivieren:

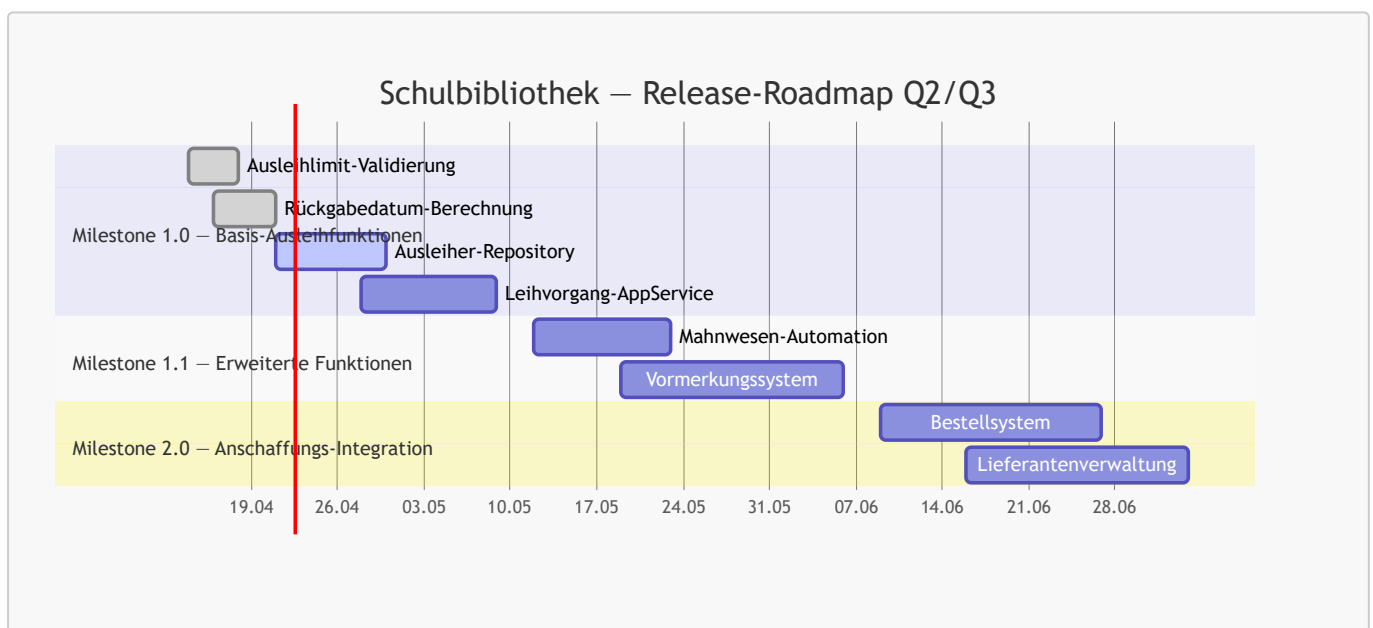
Marker	Bedeutung
<b>Milestones</b>	Zeigt Milestone-Fälligkeitsdaten als vertikale Linie

Marker	Bedeutung
<b>Iterations</b>	Zeigt Sprint-Grenzen auf der Zeitachse
<b>Item dates</b>	Zeigt die Datumsfelder der Items selbst

### Best Practices für die Roadmap im Schulprojekt

Best Practice	Begründung
<b>Start- und Zieldatum für alle Issues setzen</b>	Nur Issues mit Datumsfeldern erscheinen als Balken
<b>Zoom je nach Zweck wählen</b>	Month für Sprint-Review, Quarter für Meilensteinplanung
<b>Milestones als Marker aktivieren</b>	Macht Release-Ziele sofort sichtbar
<b>Nach Bounded Context gruppieren</b>	Zeigt, welche Domänenbereiche sich zeitlich überschneiden
<b>Regelmäßig im Sprint-Planning nutzen</b>	Die Roadmap schafft gemeinsames Verständnis im Team
<b>Dragging nur bewusst einsetzen</b>	Drag & Drop ändert Felder direkt — nicht versehentlich ziehen

### Beispiel: Roadmap für die Schulbibliothek (Quartal-Ansicht)



**Merksatz:** Die Roadmap-Ansicht ergänzt das Kanban-Board perfekt: Das Board zeigt **wo** eine Aufgabe im Prozess steht, die Roadmap zeigt **wann** sie stattfindet. Beide Ansichten zeigen dieselben Issues — aus unterschiedlichen Perspektiven.



**Achtung:** Felder wie **Start date** und **Target date** müssen als **Datums-Custom-Fields** im Projekt existieren und bei den Issues befüllt sein. Ohne diese Felder sind Issues in der Roadmap nicht als Balken sichtbar.

## Quellen

- [Customizing the roadmap layout - GitHub Docs](#)
- [About date fields - GitHub Docs](#)
- [Changing the layout of a view - GitHub Docs](#)

### 7.2.3 GitHub Issues als zentrale Aufgabenverwaltung

Issues sind das zentrale Werkzeug zur Aufgabenverwaltung in GitHub. Jedes Issue repräsentiert eine Aufgabe, einen Fehler, eine User Story oder eine technische Anforderung — und verknüpft Planung, Code und Diskussion an einem Ort.

Für die Schulbibliothek verwenden wir Issues für folgende Typen:

Issue-Typ	Beschreibung	Beispiel
<b>User Story</b>	Funktionale Anforderung aus Nutzerperspektive	<i>Als Schüler möchte ich Bücher online suchen</i>
<b>Technical Task</b>	Technische Implementierungsaufgabe	<i>Implement AusleihExemplarRepository interface</i>
<b>Bug</b>	Fehler im bestehenden Code	<i>Rückgabedatum wird falsch berechnet</i>
<b>Refactoring</b>	Code-Verbesserung ohne neue Funktion	<i>Domain Services nach Clean Architecture trennen</i>
<b>Enhancement</b>	Erweiterung bestehender Features	<i>Suchfilter um Autor-Suche erweitern</i>
<b>Documentation</b>	Fehlende oder veraltete Dokumentation	<i>API-Dokumentation für Ausleih-Endpunkte</i>

#### Labels einrichten

Labels helfen dabei, Issues schnell zu kategorisieren und zu filtern. GitHub stellt Standard-Labels bereit, die wir für das Schulprojekt anpassen:

bug	→ Fehler im Code (Farbe: Rot)
feature	→ Neue Funktion (Farbe: Blau)
user-story	→ User Story (Farbe: Lila)
technical-task	→ Technische Aufgabe (Farbe: Grau)
documentation	→ Dokumentation (Farbe: Hellblau)
refactoring	→ Code-Verbesserung (Farbe: Orange)
priority-high	→ Hohe Priorität (Farbe: Dunkelrot)
priority-low	→ Niedrige Priorität (Farbe: Hellgrün)
needs-refinement	→ Noch nicht ausreichend detailliert (Farbe: Gelb)
ausleih-kontext	→ Betrifft den Ausleih-Bounded-Context (Farbe: Türkis)

#### Labels in GitHub erstellen:

1. Repository → **Issues** → **Labels** → **New label**

2. Name, Farbe und optionale Beschreibung eingeben
3. **"Create label"** klicken

## Issues erstellen — Schritt-für-Schritt

1. **Navigation:** Repository → Tab **"Issues"** → **"New issue"**
2. **Template wählen:** Falls definiert, erscheint eine Template-Auswahl (siehe 7.2.4)
3. **Formular ausfüllen:**
  - **Titel:** Kurz und prägnant (z.B. *"Ausleihlimit-Validierung implementieren"*)
  - **Beschreibung:** Detaillierte Erklärung mit Markdown-Formatierung
  - **Labels:** Kategorisierung zuweisen
  - **Milestone:** Optional einem Release oder Sprint zuordnen
  - **Assignees:** Verantwortliche Personen zuweisen
  - **Projects:** Issue dem Project Board hinzufügen
  - **Priority / Size / Estimate / Start date / Target date:** GitHub zeigt standardmäßig diese Felder in der Seitenleiste an, wenn das Repository ein Projektboard oder Issue-Formular verwendet. Sie helfen bei Priorisierung, Aufwandsschätzung und der zeitlichen Planung.
4. **"Submit new issue"** — das Issue erhält automatisch eine Nummer (z.B. **#23**)



**Hinweis:** Nutze die Felder **Priority** und **Size**, um Issues nach Dringlichkeit und Umfang zu sortieren. **Estimate**, **Start date** und **Target date** liefern zusätzlich Planungsklarheit für Scrumban und Sprint-Reviews.

## Beispiel-Issue für eine User Story:

Titel: [User Story] Buchsuche mit Verfügbarkeitsprüfung

Labels: user-story, ausleih-kontext, needs-refinement

### ## User Story

Als **\*\*Schüler\*\*** möchte ich online die Verfügbarkeit eines Buches prüfen können, um zu wissen, ob es sich lohnt, zur Bibliothek zu gehen.

### ## Akzeptanzkriterien

- [ ] Suche nach Titel möglich
- [ ] Suche nach Autor möglich
- [ ] Anzeige des Verfügbarkeitsstatus (verfügbar/ausgeliehen/vorgemerkt)
- [ ] Anzeige des Standorts (Signatur)
- [ ] Bei ausgeliehenen Büchern: Anzeige des voraussichtlichen Rückgabedatums

### ## Technische Hinweise

- Betrifft `Ausleih-Kontext` und `Anschaffungs-Kontext`
- `AusleihExemplar` für Verfügbarkeit
- `BuchkatalogEintrag` für Titel/Autor-Suche
- Siehe DDD-Modell: Kapitel 4.1.5.4

### ## Aufwandsschätzung

Story Points: 5



**Merksatz:** Ein gut strukturiertes Issue enthält immer: **Was** soll gemacht werden (Titel), **Warum** es wichtig ist (User Story / Beschreibung), **Wann** es fertig ist (Akzeptanzkriterien) und **Wie aufwändig** es ist (Story Points).

## Quellen

- [About issues - GitHub Docs](#)
- [Managing labels - GitHub Docs](#)

## 7.2.4 Issue-Templates definieren und konfigurieren

Issue-Templates standardisieren die Erstellung von Issues und stellen sicher, dass alle wichtigen Informationen erfasst werden. Sie sparen Zeit, verbessern die Qualität und helfen besonders neuen Team-Mitgliedern beim Einstieg.

### Einrichtung von Issue-Templates

#### Methode 1: Über die GitHub-Oberfläche (empfohlen für Einsteiger)

1. Repository → **Settings** → **Features** → **Issues** → "Set up templates"
2. Vordefiniertes Template wählen (Bug Report, Feature Request) oder eigenes erstellen
3. Im Editor anpassen
4. "**Propose changes**" → committen

#### Methode 2: Manuelle Erstellung im Repository

Templates werden im Verzeichnis `.github/ISSUE_TEMPLATE/` als Dateien angelegt.

### Unterstützte Template-Formate

GitHub unterstützt zwei Formate:

#### Format 1: Markdown-Templates mit YAML-Frontmatter (`.md`)

Einfach zu erstellen, freies Markdown — ideal für Teams mit hoher Eigenverantwortung.

```

name: 📖 User Story
about: Neue User Story für Feature-Entwicklung
title: "[User Story] "
labels: user-story, needs-refinement
assignees: ''

User Story

Als **[Rolle]** möchte ich **[Ziel/Wunsch]**, um **[Nutzen]**.
```

## ## Akzeptanzkriterien

- [ ] Kriterium 1
- [ ] Kriterium 2

## ## Betroffener Bounded Context

- [ ] Ausleih-Kontext (Core Domain)
- [ ] Anschaffungs-Kontext (Supporting)
- [ ] Nutzerprofil-Kontext (Generic)
- [ ] Infrastruktur

## ## Aufwandsschätzung

Story Points: <!-- 1, 2, 3, 5, 8, 13 -->

## ## Abhängigkeiten

<!-- Optional: Verweise auf andere Issues mit #Nummer -->

### Format 2: YAML-basierte Issue-Forms (.yaml)

Strukturierte Eingabeformulare mit Pflichtfeldern und Dropdowns — ideal wenn konsistente Eingaben erzwungen werden sollen.

```
name: 📖 User Story
description: Neue User Story für Feature-Entwicklung
title: "[User Story] "
labels: ["user-story", "needs-refinement"]
body:
 - type: textarea
 id: user-story
 attributes:
 label: User Story
 description: Format – "Als [Rolle] möchte ich [Ziel], um [Nutzen]"
 placeholder: Als Schüler möchte ich Bücher online suchen können...
 validations:
 required: true

 - type: textarea
 id: acceptance-criteria
 attributes:
 label: Akzeptanzkriterien
 validations:
 required: true

 - type: dropdown
 id: bounded-context
 attributes:
 label: Betroffener Bounded Context
 options:
 - Ausleih-Kontext (Core Domain)
```

```

- Anschaffungs-Kontext (Supporting)
- Nutzerprofil-Kontext (Generic)
- Infrastruktur
validations:
 required: true

- type: dropdown
 id: story-points
 attributes:
 label: Story Points
 options: ["1", "2", "3", "5", "8", "13", "21"]

```

### Vergleich der beiden Formate:

Kriterium	Markdown-Templates (.md)	YAML-basierte Issue-Forms (.yaml)
<b>Flexibilität</b>	Sehr hoch — freies Markdown	Strukturiert durch Feldtypen
<b>Validierung</b>	Keine — Nutzer kann alles ändern	Pflichtfelder und Dropdowns möglich
<b>Benutzerfreundlichkeit</b>	Einfach für erfahrene Nutzer	Geführte Eingabe für alle
<b>Feldtypen</b>	Nur Freitext	Textarea, Dropdown, Checkboxes, Input
<b>Konsistenz</b>	Abhängig von Disziplin	Erzwungen durch Formular-Struktur
<b>Komplexität</b>	Einfache YAML-Frontmatter	Komplexere YAML-Syntax
<b>Empfohlen für</b>	Teams mit Erfahrung	Einsteiger-Teams, Schulprojekte



**Best Practice:** Beginnen Sie mit Markdown-Templates für den schnellen Einstieg. Wechseln Sie zu YAML-basierten Forms, wenn Sie strengere Validierung benötigen. Beide Formate können parallel im selben Repository verwendet werden.

### Template-Konfiguration (config.yml)

Mit einer zentralen Konfigurations-Datei steuern Sie das Verhalten der Templates:

`.github/ISSUE_TEMPLATE/config.yml:`

```

blank_issues_enabled: false # Deaktiviert Issues ohne Template
contact_links:
 - name: 📖 Dokumentation
 url: https://github.com/schule/bibliothek/wiki
 about: Bitte prüfen Sie zuerst unsere Dokumentation
 - name: 💬 Diskussionen
 url: https://github.com/schule/bibliothek/discussions
 about: Für Fragen nutzen Sie bitte die Discussions

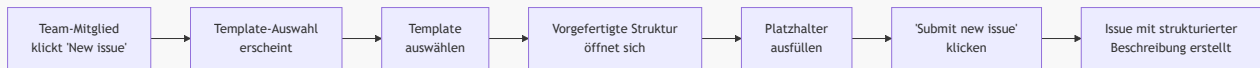
```



**Achtung:** Mit `blank_issues_enabled: false` können Nutzer **kein** Issue ohne Template erstellen. Das erzwingt konsistente Eingaben, kann aber auch einschränkend wirken. Für Schulprojekte empfiehlt sich diese Option, um Qualität sicherzustellen.

## Verwendung von Issue-Templates im Team

Nach der Einrichtung funktioniert die Nutzung wie folgt:



## Quellen

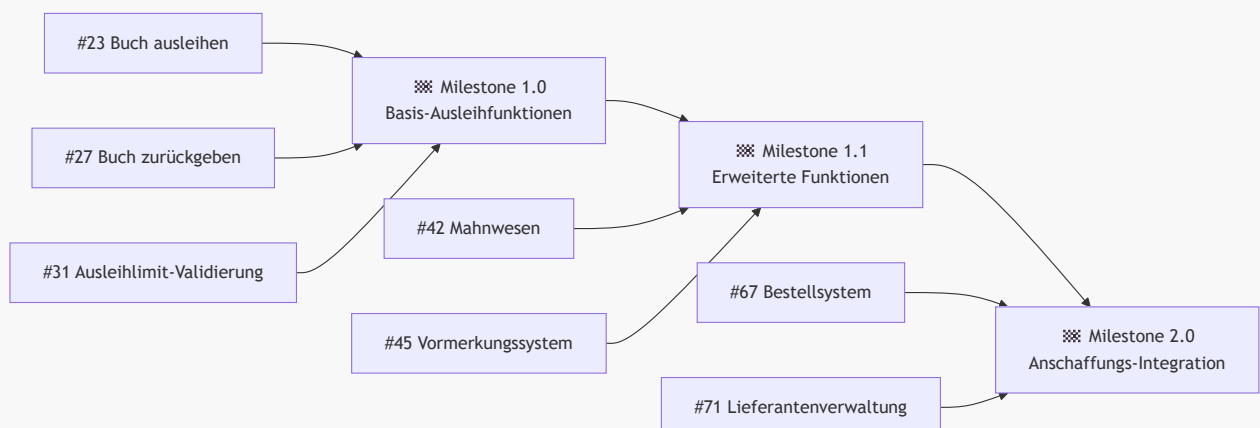
- [Configuring issue templates for your repository - GitHub Docs](#)
- [Syntax for issue forms - GitHub Docs](#)

## 7.2.5 Milestones und Release-Planung

Milestones helfen dabei, größere Lieferungen und Projektphasen in GitHub transparent zu planen. Sie gruppieren Issues und Pull Requests unter einem gemeinsamen Ziel und ermöglichen eine einfache Fortschrittsverfolgung über eine automatische Prozentanzeige.

Stellen Sie sich Milestones wie Etappenziele einer Wanderung vor: Jeder Etappenabschluss (Release) ist ein Milestone. Die einzelnen Issues sind die Schritte dazwischen. Erst wenn alle Schritte einer Etappe abgeschlossen sind, gilt der Milestone als erreicht.

### Milestone-Planung für die Schulbibliothek



## Milestone anlegen in GitHub

1. Repository → Tab **"Issues"** → **"Milestones"** → **"New milestone"**

2. **Titel** eingeben, z.B. **Release 1.0: Basis-Ausleihfunktionen**
3. Optional: **Beschreibung** mit den wichtigsten Zielen des Milestones
4. Optional: **Fälligkeitsdatum** setzen
5. **"Create milestone"** klicken

### Issues einem Milestone zuordnen

- Öffne ein Issue → Seitenleiste → **"Milestone"** → gewünschten Milestone auswählen
- Oder: Beim Erstellen des Issues direkt unter "Milestone" zuordnen
- Oder: Mehrere Issues auf einmal zuordnen über die Issues-Listenansicht (Checkbox + "Milestone" Dropdown)

### Tipps für effektive Milestone-Planung

Tipp	Begründung
Milestones für Releases/Iterationsziele, nicht für Tasks	Granularität passt — Issues sind die Tasks
Beschreibende Titel verwenden	<b>Release 1.0: Basissystem</b> ist besser als <b>v1.0</b>
Fälligkeitsdaten setzen	Erzeugt Verbindlichkeit für das Team
Regelmäßig Fortschritt prüfen	Prozentanzeige zeigt auf einen Blick, wie weit der Sprint ist
Nicht zu viele offene Milestones gleichzeitig	Max. 2-3 aktive Milestones — sonst verliert das Team den Fokus



**Merksatz:** Milestones verbinden Planung und Realität: Die Prozentanzeige zeigt jederzeit ehrlich, wie viel vom Sprint noch übrig ist. Das macht Milestone-Überprüfungen im Daily-Standup besonders wertvoll.

### Quellen

- [About milestones - GitHub Docs](#)
- [Creating and editing milestones for issues and pull requests - GitHub Docs](#)

## 7.3 Entwicklungsworkflow mit GitHub Flow

In diesem Abschnitt betrachten wir den täglichen Entwicklungsworkflow: Wie wählen wir eine Branching-Strategie? Wie benennen wir Branches konsistent? Wie funktionieren Pull Requests und Code Reviews? Und wie verknüpfen wir Issues, Branches und Pull Requests zu einem durchgängigen Workflow?

Stellen Sie sich den Entwicklungsworkflow wie eine Montagelinie vor: Jede Aufgabe (Issue) durchläuft klar definierte Stationen — vom Backlog über die Entwicklung (Feature-Branch) zur Qualitätskontrolle (Code Review via Pull Request) bis zur Auslieferung (Merge in `main`).

### Lernziele dieses Abschnitts:

- Die drei wichtigsten Branching-Strategien kennen und vergleichen können
  - GitHub Flow für das Schulprojekt begründet auswählen
  - Branch-Namen nach Konvention vergeben
  - Pull Requests strukturiert erstellen und reviewen
  - Issues, Branches und PRs durchgängig verknüpfen
- 

### 7.3.1 Branching-Strategien im Vergleich

Eine **Branching-Strategie** legt fest, wie das Team Branches erstellt, benennt, zusammenführt und verwaltet. Sie beeinflusst direkt, wie schnell Features integriert werden, wie stabil der `main`-Branch ist und wie komplex die tägliche Arbeit ist.

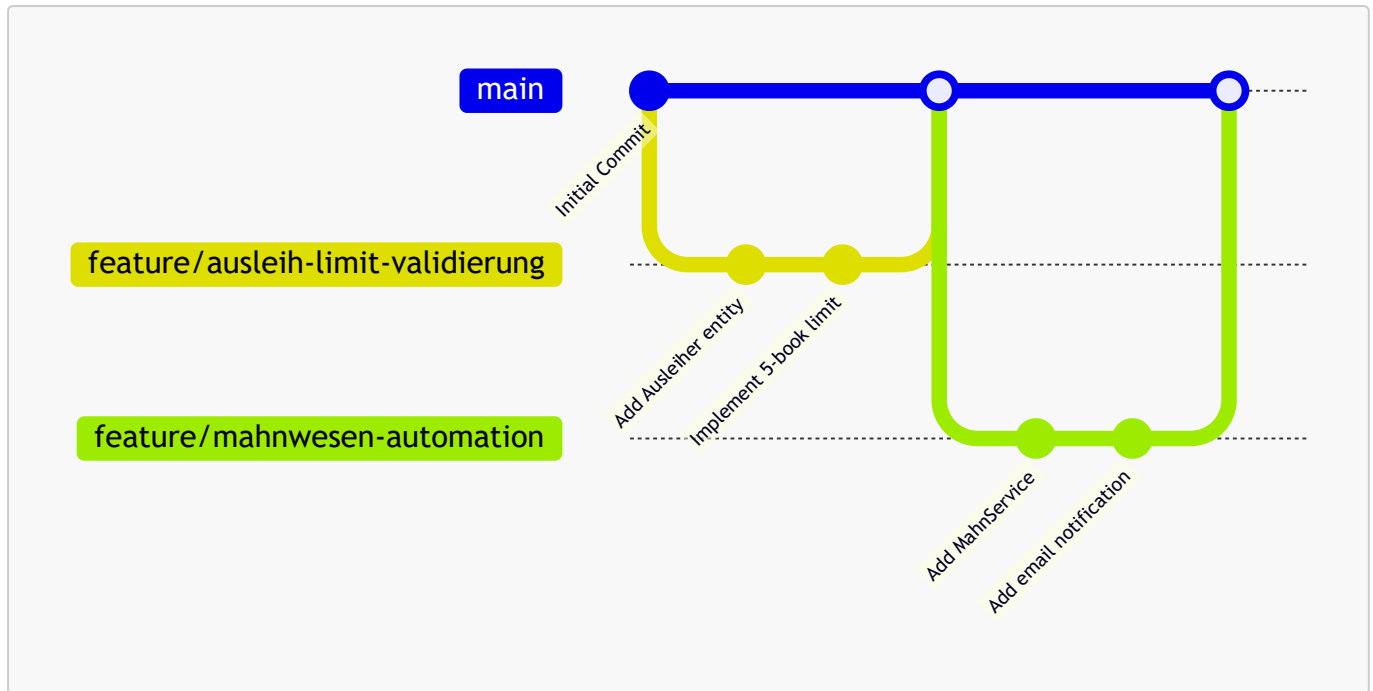
Wir stellen drei etablierte Strategien vor:

#### GitHub Flow

GitHub Flow ist eine schlanke Strategie für kontinuierliche Deployments. Sie basiert auf einem einzigen stabilen `main`-Branch und kurzlebigen Feature-Branches.

#### Branch-Struktur:

- `main` — immer stabil und deploybar
- `feature/*` — kurzlebige Entwicklungs-Branche, werden via PR gemergt
- Keine Releases- oder Hotfix-Branche

**Geeignet wenn:**

- ✓ Kontinuierliche Deployments (mehrmals pro Woche/Tag)
- ✓ Web-Anwendungen, SaaS, APIs
- ✓ Kleine bis mittlere Teams (2–10 Entwickler)

**Nicht geeignet wenn:**

- ✗ Mehrere parallele Versionen gepflegt werden müssen
- ✗ Geplante, quartalsweise Release-Termine
- ✗ App-Store-Releases mit langen Review-Prozessen



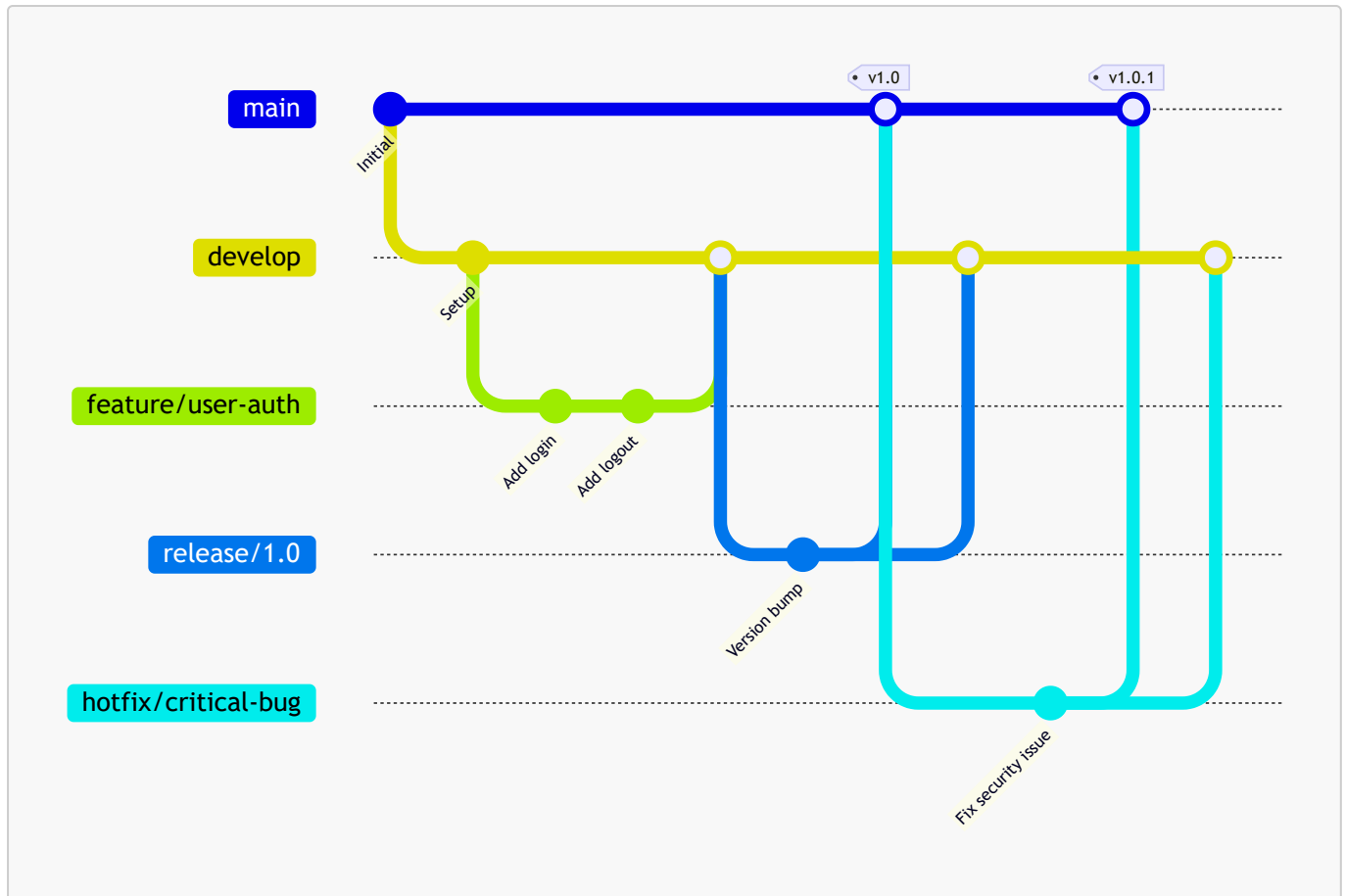
**Achtung:** Vermeiden Sie lange Feature-Branches! Je länger ein Branch von **main** getrennt ist, desto schwieriger wird das spätere Mergen. Features sollten klein gehalten und mindestens wöchentlich integriert werden.

**Git Flow**

Git Flow ist eine umfangreichere Strategie für geplante Release-Zyklen mit klar definierten Branch-Typen und einer festen Hierarchie.

**Branch-Struktur:**

- **main** — Produktions-Code, immer stabil
- **develop** — Integrations-Branch für Features
- **feature/\*** — wird in **develop** gemergt
- **release/\*** — Release-Vorbereitung
- **hotfix/\*** — dringende Production-Fixes von **main**

**Geeignet wenn:**

- ✓ Geplante Release-Zyklen (monatlich/quartalsweise)
- ✓ Mehrere Versionen parallel (v1.x und v2.x gleichzeitig)
- ✓ Enterprise-Software, Desktop- oder Mobile Apps

**Nicht geeignet wenn:**

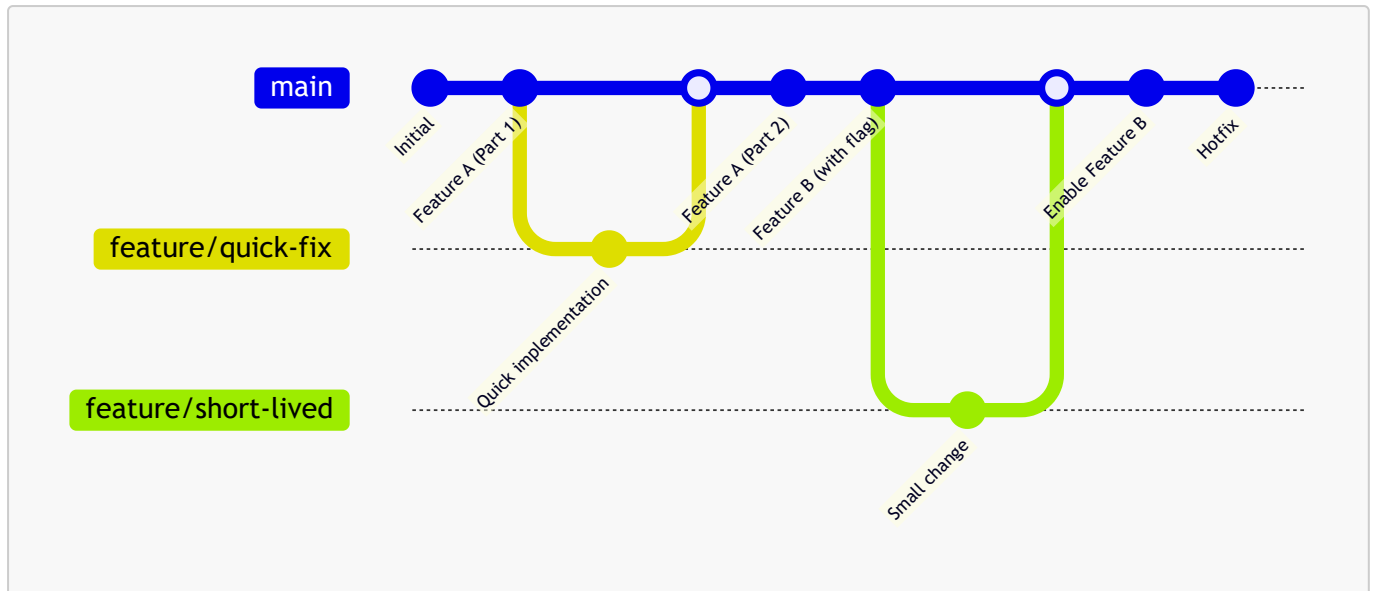
- ✗ Continuous Deployment — zu komplex für häufige Deployments
- ✗ Kleine Teams — zu viel Overhead durch viele Branches

**Trunk-Based Development**

Trunk-Based Development setzt auf minimale Branch-Komplexität und maximal häufige Integration direkt auf **main**.

**Kernprinzipien:**

- Direkter Commit auf **main** (erfahrene Teams) oder sehr kurzlebige Branches (max. 1–2 Tage)
- Unfertige Features werden per **Feature Flags** deaktiviert
- Umfangreiche Test-Automatisierung ist Pflicht



### Geeignet wenn:

- ✓ Continuous Deployment (mehrmals täglich)
- ✓ Kleine, erfahrene Teams mit hoher Disziplin
- ✓ Microservices mit unabhängigen Deployments

### Nicht geeignet wenn:

- ✗ Unerfahrene Teams — Risiko von broken builds ist hoch
- ✗ Komplexe Features mit Feature Flags zu verwalten

### Vergleich der drei Strategien

Kriterium	GitHub Flow	Git Flow	Trunk-Based
<b>Komplexität</b>	Niedrig	Hoch	Sehr niedrig
<b>Branch-Anzahl</b>	2–5 aktive	5–15 aktive	0–3 aktive
<b>Release-Zyklus</b>	Kontinuierlich	Geplant (Wochen/Monate)	Kontinuierlich (täglich)
<b>Team-Größe</b>	2–10	5–20+	2–8, erfahren
<b>Lernkurve</b>	Flach	Steil	Mittel
<b>Code-Review</b>	Ja (Pull Requests)	Ja (Pull Requests)	Optional
<b>Geeignet für</b>	Web-Apps, SaaS	Enterprise, Mobile	Microservices, Startups

### Entscheidung für die Schulbibliothek: GitHub Flow

Wir wählen **GitHub Flow** aus folgenden Gründen:

Faktor	Begründung
<b>Schüler-Team</b>	Einfach zu erlernen, klare Regeln: Branch → PR → Review → Merge
<b>Projektgröße</b>	3–5 Entwickler, Web-Anwendung, keine parallelen Versionen

Faktor	Begründung
<b>Scrumban</b>	2-wöchige Sprints passen zu wöchentlicher Feature-Integration
<b>Qualitätssicherung</b>	PRs erzwingen Code-Reviews — pädagogisch wertvoll
<b>GitHub-Integration</b>	Projects, Actions und Issues optimal für GitHub Flow ausgelegt



**Merksatz:** Die Branching-Strategie ist kein Dogma, sondern ein Werkzeug. GitHub Flow bietet für das Schulprojekt die beste Balance zwischen Einfachheit, Qualitätssicherung und Lerneffekt. Sie kann später angepasst werden, wenn sich Anforderungen oder Team-Reife ändern.

## Quellen

- [GitHub Flow - GitHub Docs](#)
- [Gitflow Workflow \(Atlassian\)](#)
- [Trunk-Based Development \(trunkbaseddevelopment.com\)](#)

## 7.3.2 Branch-Naming-Konventionen

Konsistente Branch-Namen helfen dem Team, den Zweck eines Branches auf einen Blick zu erfassen und erleichtern die Navigation, das Filtern und die CI/CD-Automatisierung.

### Präfixe nach Zweck

Präfix	Verwendung	Beispiel
<code>feature/</code>	Neue Funktionen	<code>feature/buchsuche-api</code>
<code>bugfix/</code>	Fehlerbehebungen	<code>bugfix/ausleihdatum-anzeige</code>
<code>hotfix/</code>	Dringende Production-Fixes	<code>hotfix/login-absturz</code>
<code>refactor/</code>	Code-Verbesserungen ohne neue Funktion	<code>refactor/repository-cleanup</code>
<code>test/</code>	Test-Ergänzungen	<code>test/mahnservice-unit-tests</code>
<code>doc/</code>	Dokumentations-Updates	<code>doc/api-anleitung-aktualisieren</code>

### Regeln für Branch-Namen

- ✓ **Bindestriche** (-) zur Worttrennung — keine Leerzeichen, keine Unterstriche
- ✓ **Issue-Nummer** integrieren — verknüpft Branch direkt mit der Aufgabe
- ✓ **Kurz und prägnant** — fokussiert auf die Hauptaufgabe
- X Generische Begriffe wie `update`, `changes`, `stuff` vermeiden

### Namenskonvention für die Schulbibliothek

feature/[issue-nr]-[kurzbeschreibung]	feature/42-mahnwesen-automation
bugfix/[issue-nr]-[kurzbeschreibung]	bugfix/57-rueckgabedatum-fehler
hotfix/[kurzbeschreibung]	hotfix/login-absturz
refactor/[kurzbeschreibung]	refactor/domain-services-optimieren
doc/[kurzbeschreibung]	doc/api-dokumentation-erweitern



**Merksatz:** GitHub erkennt Issue-Nummern in Branch-Namen automatisch und zeigt die Verknüpfung in der Issue-Ansicht an. Ein Branch `feature/42-mahnwesen` erscheint direkt bei Issue `#42` — ohne manuelles Verknüpfen.



**Praxis-Tipp:** Dokumentieren Sie die Namenskonvention in der `CONTRIBUTING.md` (siehe Kapitel 7.1.2). So können neue Team-Mitglieder sofort nach Konvention arbeiten, ohne fragen zu müssen.

---

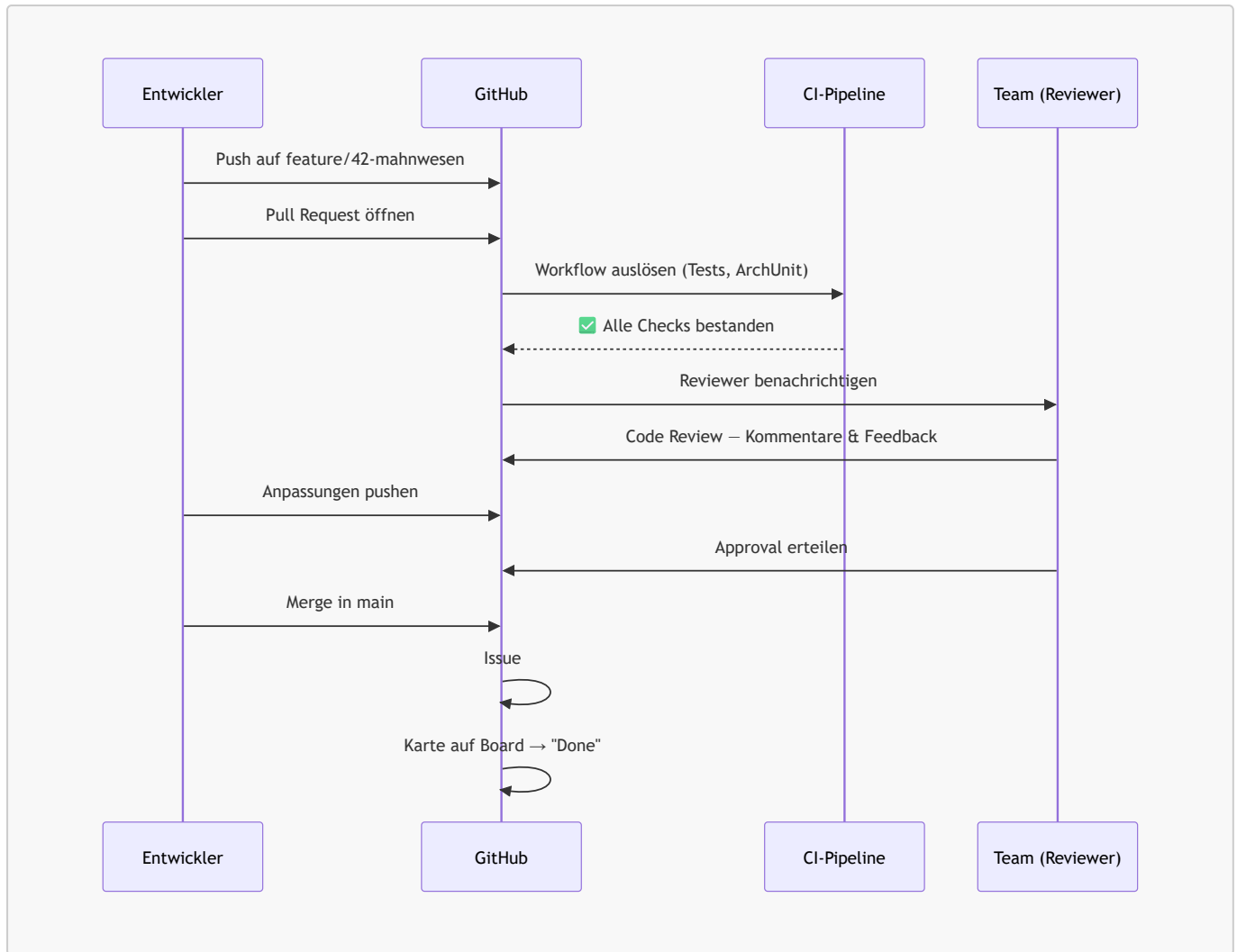
## Quellen

- [GitHub Branching Name Best Practices \(dev.to\)](#)
- 

### 7.3.3 Pull Requests und Code Reviews

Änderungen werden nicht direkt auf `main` gemacht, sondern in Feature-Banches entwickelt. Nach Fertigstellung wird via **Pull Request (PR)** eine Integration in `main` vorgeschlagen. Der PR ist gleichzeitig Qualitätstor, Diskussionsplattform und Dokumentation der Änderung.

#### Der Pull-Request-Workflow



## Praktisches Beispiel: Feature "Ausleihlimit"

**Schritt 1 — Issue erstellen:** #12 Als System soll sichergestellt sein, dass ein Schüler maximal 5 Bücher ausleihen kann

**Schritt 2 — Branch erstellen:** feature/12-ausleih-limit-validierung

**Schritt 3 — Implementierung mit aussagekräftigen Commits:**

```
git commit -m "feat(domain): Add Ausleiher aggregate with loan limit"
git commit -m "feat(domain): Throw AusleihLimitErreichtException at 5 books"
git commit -m "test(domain): Add unit tests for loan limit validation"
```

**Schritt 4 — Implementierungsbeispiel (Domain-Layer):**

```
// domain/aggregates/Ausleiher.java
public class Ausleiher {
 private static final int MAX_AUSLEIHEN_SCHUELER = 5;

 private final AusleiherId id;
 private final Rolle rolle;
 private final List<Ausleihe> ausleihen = new ArrayList<>();
```

```

 public void leiheBuchAus(AusleihExemplar exemplar) {
 if (rolle == Rolle.SCHUELER && ausleihen.size() >= MAX_AUSLEIHEN_SCHUELER)
 {
 throw new AusleihLimitErreichtException(
 String.format("Schüler dürfen maximal %d Bücher ausleihen",
 MAX_AUSLEIHEN_SCHUELER)
);
 }
 ausleihen.add(new Ausleihe(exemplar, LocalDate.now().plusWeeks(2)));
 }
}

```

**Schritt 5 — Pull Request öffnen** mit Titel `feat(domain): Implement 5-book loan limit for students` und `Closes #12` in der Beschreibung



**Merksatz:** `Closes #12` in der PR-Beschreibung bewirkt, dass Issue `#12` beim Merge **automatisch geschlossen** wird — keine manuelle Nacharbeit notwendig.

### Tipps für gute Code Reviews

Für Reviewer	Für den PR-Ersteller
Konstruktives Feedback — nicht persönlich	PR klein halten (max. 400 Zeilen)
Konkrete Verbesserungsvorschläge geben	Beschreibung ausfüllen, Kontext erklären
Fragen stellen statt Befehle erteilen	Auf Feedback eingehen, nicht verteidigen
Architektur-Prinzipien prüfen (DDD, Clean)	Tests mitliefern
Zeitnah reviewen (innerhalb 24h)	Reviewer explizit zuweisen

### Quellen

- [About pull requests - GitHub Docs](#)
- [About pull request reviews - GitHub Docs](#)

### 7.3.4 PR-Templates definieren

Ähnlich wie Issue-Templates (siehe 7.2.4) sorgen **Pull-Request-Templates** dafür, dass jeder PR eine konsistente, vollständige Beschreibung enthält. Das Template wird automatisch in das Beschreibungsfeld eingefügt, wenn ein Entwickler einen neuen PR öffnet.

### Speicherort und Einrichtung

GitHub sucht das PR-Template an folgenden Orten (Priorität von oben nach unten):

Speicherort	Dateiname
Repository-Root	<code>pull_request_template.md</code>
<code>docs/</code> Verzeichnis	<code>docs/pull_request_template.md</code>
<code>.github/</code> Verzeichnis (empfohlen)	<code>.github/pull_request_template.md</code>

### Datei anlegen:

```
Lokal im Repository-Verzeichnis
mkdir -p .github
Datei erstellen und im Editor befüllen
```

Oder über GitHub-UI: **"Add file"** → **"Create new file"** → Dateiname `.github/pull_request_template.md` eingeben.

### PR-Template für die Schulbibliothek

```
Beschreibung
<!-- Kurze Zusammenfassung: Was wurde implementiert/geändert und warum? -->

Verknüpftes Issue
Closes #

Typ der Änderung
- [] 🐛 Bug-Fix
- [] ✨ Neues Feature
- [] 💥 Breaking Change
- [] 📖 Dokumentation
- [] ♻️ Refactoring
- [] 🧪 Tests

Betroffener Bounded Context
- [] Ausleih-Kontext (Core Domain)
- [] Anschaffungs-Kontext (Supporting)
- [] Nutzerprofil-Kontext (Generic)
- [] Infrastruktur

Checkliste vor dem Review
- [] ✅ Code kompiliert fehlerfrei (`mvn clean compile`)
- [] 🧪 Unit-Tests geschrieben/aktualisiert und grün
- [] 🏠 Clean-Architecture-Prinzipien eingehalten
- [] 📖 Dokumentation aktualisiert (Javadoc, README)
- [] 🔗 Branch ist up-to-date mit `main`

Hinweise für Reviewer
<!-- Optional: Worauf sollten Reviewer besonders achten? -->
```



**Praxis-Tipp:** Das Template im `.github/`-Verzeichnis ablegen. So bleiben alle GitHub-spezifischen Konfigurationen (Workflows, Issue-Templates, PR-Template) an einem zentralen Ort und das Repository-Root bleibt übersichtlich.

Vorteil	Beschreibung
Konsistenz	Jeder PR folgt derselben Struktur
Vollständigkeit	Wichtige Informationen werden nicht vergessen
Effizienz	Reviewer finden alle Infos schnell
Onboarding	Neue Mitglieder wissen sofort, was erwartet wird
Qualität	Checkliste erinnert an Best Practices

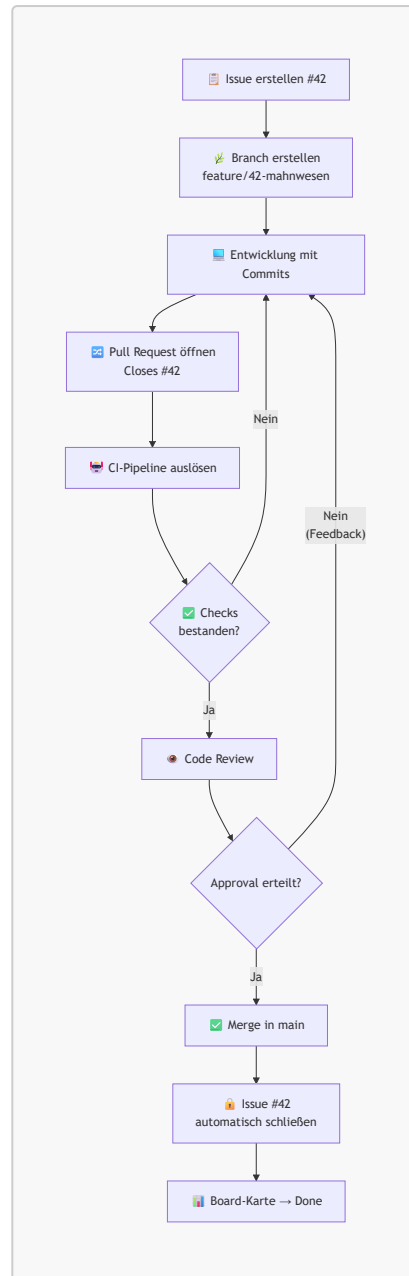
#### Quellen

- [Creating a pull request template for your repository - GitHub Docs](#)

### 7.3.5 Verbindung zwischen Issues, Branches und Pull Requests

Der vollständige GitHub-Workflow verbindet alle Werkzeuge zu einem durchgängigen Prozess: Issues bilden die Planung, Branches die Entwicklung, Commits die Änderungshistorie und Pull Requests die Qualitätssicherung.

#### Der vollständige Workflow



## Konkrete Umsetzung: Feature "Mahnwesen" (#42)

### Issue #42 anlegen:

Titel: [Feature] Automatisches Mahnwesen für überfällige Ausleihen  
 Labels: feature, ausleih-kontext  
 Milestone: 1.1 - Erweiterte Funktionen  
 Story Points: 8

**Branch direkt aus Issue erstellen** (GitHub-UI → "Create a branch"): → erzeugt automatisch `42-feature-automatisches-mahnwesen`  
 (alternativ manuell: `feature/42-mahnwesen-automation`)

### Commits nach Conventional Commits:

```
git commit -m "feat(domain): Add Mahnung entity to Ausleiher aggregate"
git commit -m "feat(domain): Implement MahnService with overdue detection"
git commit -m "test(domain): Add unit tests for Mahnstufen logic"
git commit -m "feat(infra): Add email notification for Mahnungen"
```

### Pull Request öffnen:

Titel: feat(domain): Implement automatic Mahnwesen

Beschreibung:

Implementiert das Mahnwesen aus dem DDD-Modell (Kapitel 4.1.4.3).  
Der MahnService findet täglich überfällige Ausleihen und versendet Mahnungen.

Details:

- Mahnung als Entity im Ausleiher-Aggregat
- MahnService als Domain Service
- 3 Mahnstufen mit steigenden Gebühren
- E-Mail-Versand über Infrastructure-Layer

Closes #42

### Scrumban-Sprintplanung mit GitHub Projects

Im **Scrumban-Ansatz** kombinieren wir Scrum-Elemente (Sprints, Planning) mit Kanban-Prinzipien (kontinuierlicher Flow, WIP-Limits):

#### Sprint 1 — Basis-Ausleihfunktionen (2 Wochen):

- **Sprint-Ziel:** Ein Schüler kann ein Buch ausleihen und zurückgeben
- **Kapazität:** 40 Story Points

Issue	Beschreibung	Story Points
#23	Ausleihlimit-Validierung	5 SP
#27	Rückgabedatum-Berechnung	3 SP
#31	ISBN-ValueObject	2 SP
#35	Ausleiher-Repository	8 SP
#38	AusleihExemplar-Aggregat	8 SP
#40	Leihvorgang-ApplicationService	13 SP
<b>Gesamt</b>		<b>39 SP</b>

#### WIP-Limits:

- **In Progress:** max. 3 Issues gleichzeitig

- **Code Review:** max. 4 PRs gleichzeitig
- **Testing:** max. 2 Features gleichzeitig



**Merksatz: WIP-Limits** (Work In Progress) verhindern, dass zu viele Aufgaben gleichzeitig begonnen werden. Sie zwingen das Team, Aufgaben zu beenden, bevor neue gestartet werden — das erhöht die Gesamtdurchlaufgeschwindigkeit.

**Daily-Standup mit GitHub-Board:** Das Team trifft sich täglich für 15 Minuten und nutzt das GitHub-Board als zentrale Informationsquelle:

- Was ist heute in "Testing" oder "Code Review" — gibt es Blocker?
- Welche PRs warten länger als 24h auf Review?
- Was kann von "In Progress" nach "Code Review" verschoben werden?

---

#### Quellen

- [Linking a pull request to an issue - GitHub Docs](#)
  - [About project boards - GitHub Docs](#)
-

## 7.4 CI/CD mit GitHub Actions

Stellen Sie sich vor, jedes Mal wenn ein Teammitglied einen Pull Request öffnet, überprüft ein unsichtbarer Qualitätswächter automatisch den Code: Er kompiliert das Projekt, führt alle Tests aus, misst die Code-Abdeckung und meldet das Ergebnis direkt im Pull Request. Erst wenn dieser Wächter grünes Licht gibt, darf der Code gemergt werden. Genau das leisten **GitHub Actions Workflows** in Kombination mit CI/CD-Pipelines.



**Merksatz: CI (Continuous Integration)** bedeutet, dass Code-Änderungen kontinuierlich und automatisch getestet werden, sobald sie ins Repository gepusht werden. **CD (Continuous Delivery/Deployment)** erweitert dies: Getesteter Code wird automatisch in eine Test- oder Produktionsumgebung ausgeliefert.

### Lernziele dieses Abschnitts:

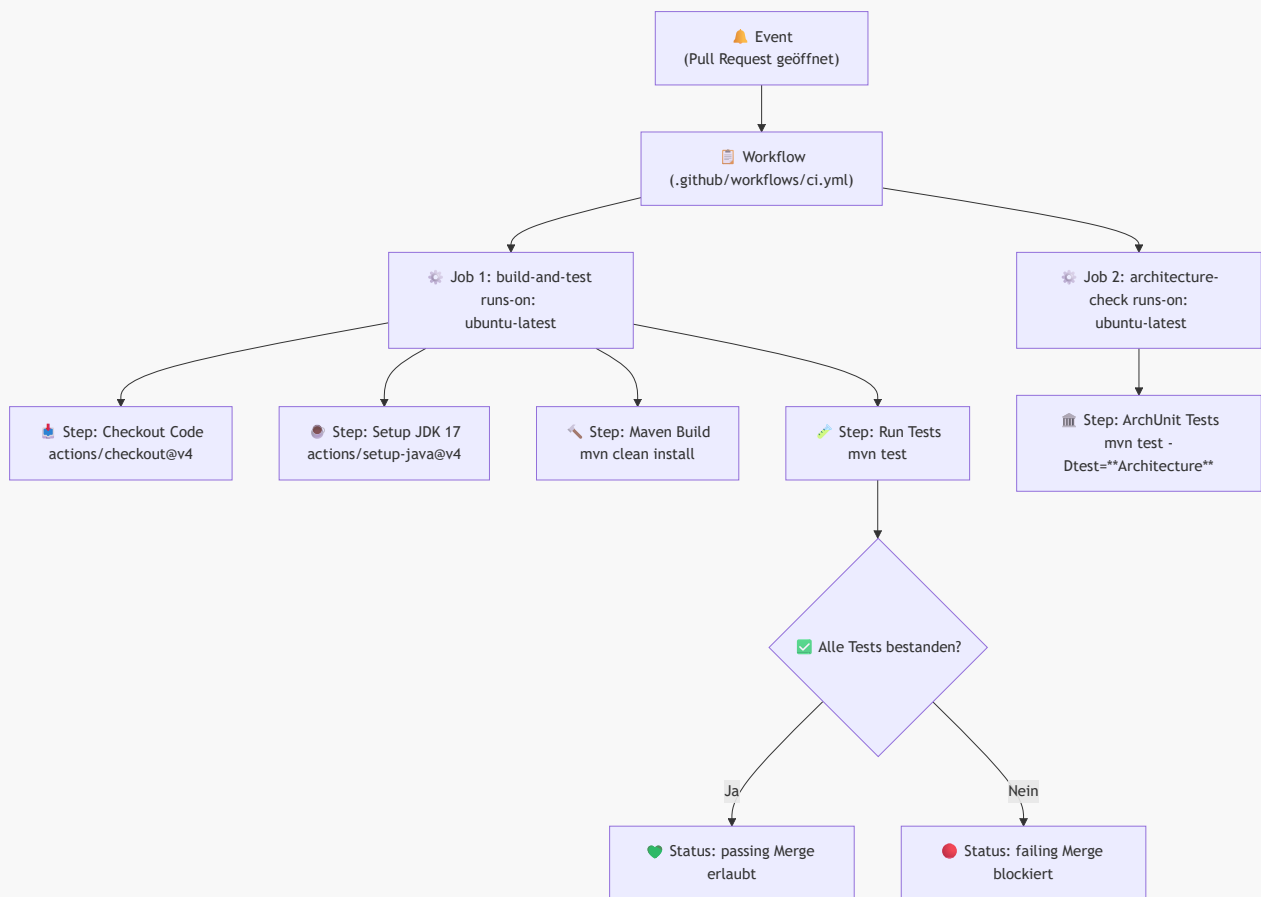
- Den Aufbau einer GitHub Actions Workflow-Datei verstehen
- Einen vollständigen CI-Workflow für Pull Requests erstellen
- Den Unterschied zwischen GitHub-hosted und Self-hosted Runnern erklären
- Service-Container für Integrationstests einsetzen

### 7.4.1 Anatomie eines GitHub Actions Workflows

GitHub Actions Workflows sind YAML-Dateien im Verzeichnis `.github/workflows/`. Sie beschreiben **wann** etwas passiert (*Trigger/Event*), **was** passiert (*Jobs mit Steps*) und **wo** es ausgeführt wird (*Runner*).

### Die fünf Kernkonzepte

Konzept	Beschreibung	Beispiel
<b>Workflow</b>	Die gesamte Automatisierungsdatei	<code>.github/workflows/ci.yml</code>
<b>Event</b>	Das auslösende Ereignis	<code>pull_request</code> , <code>push</code>
<b>Job</b>	Gruppe von Steps auf einem Runner	<code>build-and-test</code>
<b>Step</b>	Einzelner Befehl oder Action	<code>mvn clean install</code>
<b>Runner</b>	VM, die den Job ausführt	<code>ubuntu-latest</code>



## Grundstruktur einer Workflow-Datei

```

.github/workflows/ci.yml
name: CI Pipeline # Anzeigename in der GitHub-Oberfläche

on: # Trigger: Wann soll der Workflow laufen?
 pull_request:
 branches: [main]
 push:
 branches: [main]

jobs: # Was soll ausgeführt werden?
 build-and-test:
 runs-on: ubuntu-latest # Wo soll es laufen? (Runner)
 steps:
 - name: Code auschecken
 uses: actions/checkout@v4 # Fertige Action aus dem Marketplace
 - name: Build
 run: mvn clean install # Shell-Befehl direkt ausführen

```



**Vertiefung:** **uses** vs. **run**

- **uses:** Verwendet eine fertige, wiederverwendbare *GitHub Action* aus dem Marketplace (z.B. `actions/checkout@v4` lädt den Repository-Code herunter). Das `@v4` ist die Version — immer eine Version angeben für reproduzierbare Builds.
- **run:** Führt direkt einen Shell-Befehl aus (Bash unter Linux/macOS, PowerShell/cmd unter Windows).

---

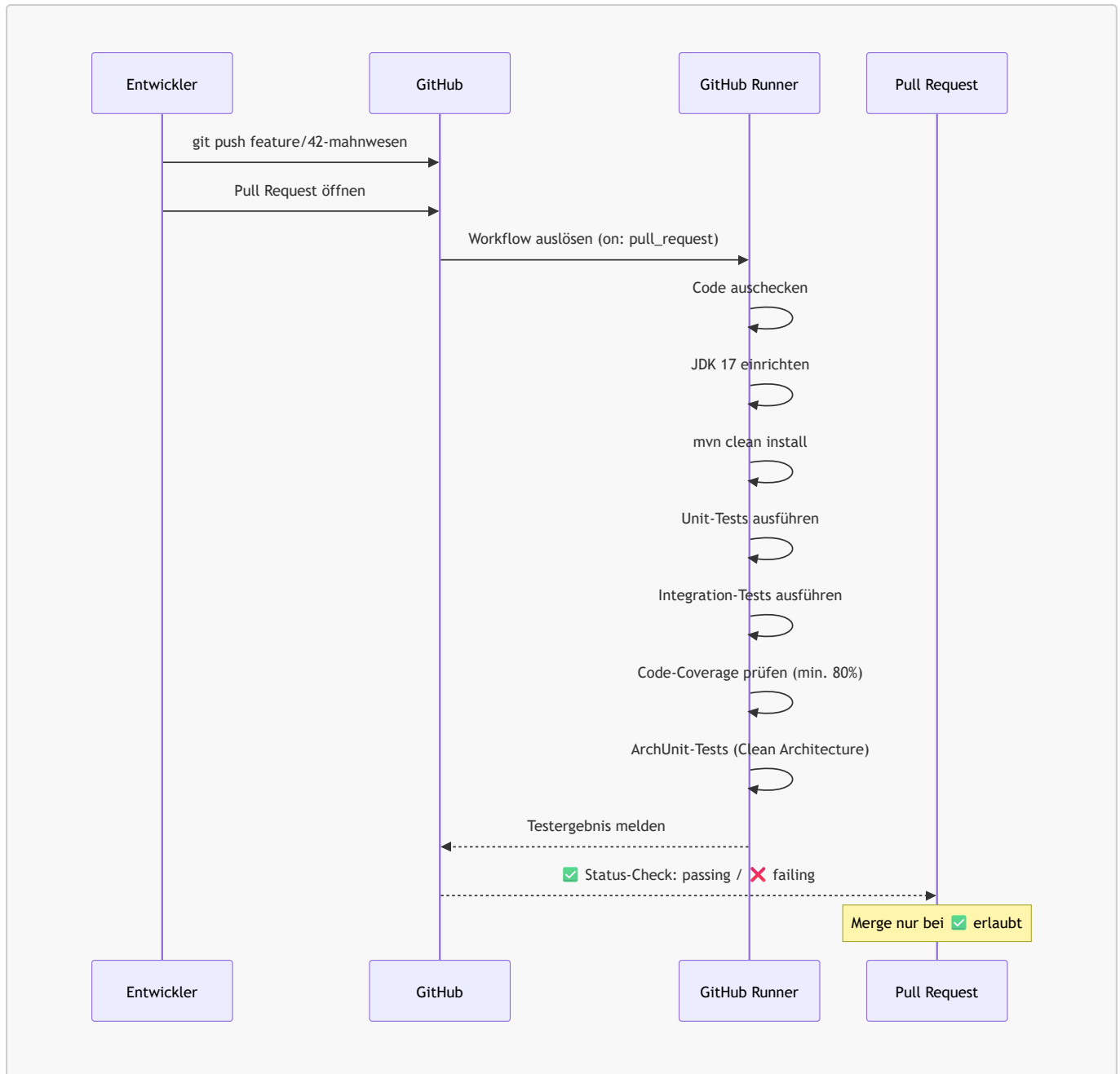
## Quellen

- [Understanding GitHub Actions - GitHub Docs](#)
  - [Workflow syntax for GitHub Actions - GitHub Docs](#)
- 
- 

## 7.4.2 CI-Workflow bei Pull Requests

Der wichtigste Einsatzort von CI-Workflows ist der **Pull Request**. Sobald ein Entwickler einen PR öffnet oder aktualisiert, startet GitHub automatisch die konfigurierte Pipeline. Das Ergebnis erscheint direkt im PR als Status-Check — der Merge-Button bleibt gesperrt, solange Checks fehlschlagen (sofern Branch Protection Rules aktiv sind, siehe Kapitel 7.1.3).

### Ablauf



## Vollständiger CI-Workflow für die Schulbibliothek

```

.github/workflows/pull-request-ci.yml
name: Pull Request CI

on:
 pull_request:
 branches: [main]
 types: [opened, synchronize, reopened]

jobs:
 # Job 1: Build und Tests
 build-and-test:
 name: 🛠️ Build & Test
 runs-on: ubuntu-latest

 steps:

```

- name:  Code auschecken  
uses: actions/checkout@v4
- name:  JDK 17 einrichten  
uses: actions/setup-java@v4  
with:  
  java-version: '17'  
  distribution: 'temurin'  
  cache: 'maven' # Maven-Abhängigkeiten cachen → schnellere

## Builds

- name:  Projekt bauen  
run: mvn clean install -DskipTests
- name:  Unit-Tests ausführen  
run: mvn test -Dtest="\*\*/unit/\*\*/\*Test"
- name:  Integrations-Tests ausführen  
run: mvn test -Dtest="\*\*/integration/\*\*/\*Test"
- name:  Code-Coverage prüfen (min. 80% im Domain-Layer)  
run: |  
  mvn jacoco:report  
  mvn jacoco:check \  
    -Djacoco.coverage.minimum=0.80 \  
    -Djacoco.includes=com/schule/bibliothek/domain/\*\*
- name:  Coverage-Report hochladen  
uses: codecov/codecov-action@v4  
with:  
  files: ./target/site/jacoco/jacoco.xml  
  fail\_ci\_if\_error: false

# Job 2: Architektur-Validierung (läuft erst nach Job 1)

## architecture-check:

name:  Clean Architecture Check  
runs-on: ubuntu-latest  
needs: build-and-test

## steps:

- name:  Code auschecken  
uses: actions/checkout@v4
- name:  JDK 17 einrichten  
uses: actions/setup-java@v4  
with:  
  java-version: '17'  
  distribution: 'temurin'  
  cache: 'maven'
- name:  ArchUnit-Tests ausführen  
run: mvn test -Dtest="\*\*/architecture/\*\*/\*Test"

## Was dieser Workflow prüft und warum

Schritt	Was wird geprüft?	Warum wichtig?
<b>Build</b>	Kompiliert der Code fehlerfrei?	Syntaxfehler werden sofort erkannt
<b>Unit-Tests</b>	Funktionieren Domain-Methoden korrekt?	Core Business-Logik ist korrekt
<b>Integration-Tests</b>	Funktionieren Repositories mit echter DB?	Datenbank-Interaktionen sind stabil
<b>Coverage</b>	Mind. 80% des Domain-Codes getestet?	Ausreichende Testabdeckung sichergestellt
<b>ArchUnit</b>	Hält Code Clean-Architecture-Regeln ein?	Domain-Layer bleibt unabhängig von Infrastructure

## CI-Checks mit Branch Protection Rules verknüpfen

Damit die CI-Checks als **Pflicht-Gate** wirken (kein Merge bei roten Tests), müssen sie in den Branch Protection Rules als Required Status Checks eingetragen werden (siehe Kapitel 7.1.3):

- Repository → **Settings** → **Rules** → Ruleset **Protect main** öffnen
- **Require status checks to pass** aktivieren
- Status-Checks hinzufügen:  **Build & Test** und  **Clean Architecture Check**

Nach erfolgreicher Konfiguration sieht ein PR mit bestandenen Checks so aus:

Pull Request: feat(domain): Implement Mahnwesen

---

Checks





Build & Test

– passed in 2m 34s





Clean Architecture Check

– passed in 1m 12s

---



All checks have passed

[ Merge pull request ]

← Button ist aktiv

## Quellen

- [Events that trigger workflows: pull\\_request - GitHub Docs](#)
- [About protected branches - GitHub Docs](#)

## 7.4.3 GitHub-hosted Runner vs. Self-hosted Runner

Workflows können auf zwei Typen von **Runnern** ausgeführt werden: auf von GitHub verwalteten Cloud-VMs oder auf einem eigenen, lokal betriebenen Rechner.

**GitHub-hosted Runner** sind virtuelle Maschinen, die GitHub für jede Workflow-Ausführung frisch bereitstellt. Nach Abschluss des Jobs wird die VM automatisch gelöscht — keine Konfiguration notwendig.

### Verfügbare GitHub-hosted Runner

Label ( <b>runs-on</b> )	Betriebssystem	CPU	RAM	Freies Kontingent
<code>ubuntu-latest</code>	Ubuntu 24.04	4 Cores	16 GB	✓ 2.000 Min./Monat (Free)
<code>ubuntu-22.04</code>	Ubuntu 22.04	4 Cores	16 GB	✓ enthalten
<code>windows-latest</code>	Windows Server 2022	4 Cores	16 GB	✓ 500 Min./Monat (Free)
<code>macos-latest</code>	macOS 14 (Apple Silicon)	3 Cores	7 GB	✓ 10 Min./Monat (Free)



**Merksatz:** Für **öffentliche** Repositories sind GitHub-hosted Runner kostenlos und unlimitiert. Für **private** Repositories gilt ein monatliches Freikontingent — für Schulprojekte in der Regel ausreichend.

**Self-hosted Runner** sind eigene Rechner (Laptop, Desktop-PC, Schul-Server), die als Runner bei GitHub registriert werden. Sie eignen sich, wenn spezielle Hardware/Software benötigt wird, Kosten gespart werden sollen oder Zugriff auf lokale Ressourcen (z.B. lokale Datenbank) notwendig ist.

### Vergleich

Kriterium	GitHub-hosted Runner	Self-hosted Runner
<b>Einrichtungsaufwand</b>	Keiner — sofort nutzbar	Einmalige Installation nötig
<b>Betriebsaufwand</b>	Keiner (GitHub kümmert sich)	Team verantwortlich für Wartung
<b>Kosten</b>	Kostenlos für Public Repos	Nur Stromkosten / eigene Hardware
<b>Isolation</b>	Frische VM bei jedem Run	Persistente Umgebung (Vorsicht: Zustand!)
<b>Flexibilität</b>	Standardumgebung, erweiterbar	Volle Kontrolle über Software/Hardware
<b>Sicherheit</b>	Sehr hoch (isolierte VM)	Eigene Verantwortung
<b>Geeignet für</b>	Standard-CI-Pipelines	Spezielle Anforderungen, lokale Ressourcen



**Sicherheitshinweis:** Self-hosted Runner dürfen **niemals für öffentliche Repositories** verwendet werden! Externe Nutzer könnten via Pull Requests Workflows mit böartigem Code auf dem Runner ausführen. Für private Schulprojekte ist dies unproblematisch.

### Quellen

- [About GitHub-hosted runners - GitHub Docs](#)
- [About self-hosted runners - GitHub Docs](#)

## 7.4.4 Self-hosted Runner lokal einrichten

Ein Self-hosted Runner auf einem lokalen Entwicklungsrechner oder Schul-Server ermöglicht schnellere Builds (kein Up-/Download-Overhead) und direkten Zugriff auf lokal installierte Datenbanken.

## Schritt-für-Schritt: Runner registrieren

### 1. GitHub-Einstellungen öffnen

- Repository → **Settings** → **Actions** → **Runners** → "New self-hosted runner"
- Betriebssystem auswählen (Linux / Windows / macOS)
- Den angezeigten Befehlen folgen — GitHub generiert einen einmaligen Registrierungs-Token

### 2. Runner-Software herunterladen und konfigurieren

*Windows (PowerShell als Administrator):*

```
mkdir C:\actions-runner; cd C:\actions-runner

Runner-Software herunterladen (Version aus GitHub-UI kopieren!)
Invoke-WebRequest -Uri
https://github.com/actions/runner/releases/download/v2.x.x/actions-runner-win-x64-
2.x.x.zip -OutFile actions-runner.zip

Entpacken
Add-Type -AssemblyName System.IO.Compression.FileSystem
[System.IO.Compression.ZipFile]::ExtractToDirectory("$PWD/actions-runner.zip",
"$PWD")

Konfigurieren (Token aus GitHub-UI kopieren!)
.\config.cmd --url https://github.com/OWNER/REPO --token ABCDEF123...
```

*Linux/macOS (Terminal):*

```
mkdir ~/actions-runner && cd ~/actions-runner

curl -o actions-runner-linux-x64.tar.gz -L \
https://github.com/actions/runner/releases/download/v2.x.x/actions-runner-linux-
x64-2.x.x.tar.gz

tar xzf ./actions-runner-linux-x64.tar.gz

Konfigurieren (Token aus GitHub-UI kopieren!)
./config.sh --url https://github.com/OWNER/REPO --token ABCDEF123...
```

### 3. Runner starten

```
./run.sh # Linux/macOS – einmaliger Start zum Testen
.\run.cmd # Windows
```

```
Als Dienst für dauerhaften Betrieb installieren
sudo ./svc.sh install && sudo ./svc.sh start # Linux/macOS
.\svc.cmd install # Windows (als Admin)
```

#### 4. Verbindung prüfen

Nach erfolgreichem Start erscheint:

```
✓ Connected to GitHub
2026-04-22 09:15:32Z: Listening for Jobs
```

Und in den Repository-Settings ist der Runner mit Status "**Idle**" (bereit) sichtbar.

#### Self-hosted Runner im Workflow verwenden

```
jobs:
 build-and-test:
 runs-on: self-hosted # statt ubuntu-latest
 # oder mit Labels:
 # runs-on: [self-hosted, schulbibliothek, java17]
```

 **Praxis-Tipp:** Vergeben Sie beim Konfigurationsschritt **Labels** (z.B. `schulbibliothek`, `java17`, `local-db`), um verschiedene Maschinen im Workflow gezielt anzusprechen: `runs-on: [self-hosted, schulbibliothek]`.

---

#### Quellen

- [Adding self-hosted runners - GitHub Docs](#)
  - [Configuring the self-hosted runner application as a service - GitHub Docs](#)
- 

#### 7.4.5 Test-Instanzen und Deployment-Previews

Neben dem reinen Testen kann GitHub Actions auch dazu genutzt werden, für jeden Pull Request automatisch eine **isolierte Test-Instanz** der Anwendung zu starten. So kann das Team oder die Bibliothekarin als Stakeholder neue Features direkt ausprobieren — ohne lokalen Code aufsetzen zu müssen.

#### Service-Container: Datenbank für Integrationstests

GitHub Actions unterstützt **Service-Container** — Docker-Container, die parallel zu einem Job laufen und z.B. eine PostgreSQL-Datenbank für Integrationstests bereitstellen:

```
.github/workflows/integration-tests.yml
name: Integration Tests mit Datenbank
```

```

on:
 pull_request:
 branches: [main]

jobs:
 integration-test:
 name:  Integration Tests
 runs-on: ubuntu-latest

 services:
 postgres:
 image: postgres:15
 env:
 POSTGRES_DB: bibliothek_test
 POSTGRES_USER: testuser
 POSTGRES_PASSWORD: testpassword
 ports:
 - 5432:5432
 options: >-
 --health-cmd pg_isready
 --health-interval 10s
 --health-timeout 5s
 --health-retries 5

 steps:
 - uses: actions/checkout@v4

 - uses: actions/setup-java@v4
 with:
 java-version: '17'
 distribution: 'temurin'
 cache: 'maven'

 - name:  Integrations-Tests mit PostgreSQL ausführen
 run: mvn test -Dtest="**/integration/**/*Test"
 env:
 SPRING_DATASOURCE_URL: jdbc:postgresql://localhost:5432/bibliothek_test
 SPRING_DATASOURCE_USERNAME: testuser
 SPRING_DATASOURCE_PASSWORD: testpassword

```

GitHub startet den PostgreSQL-Container *vor* dem eigentlichen Job. Spring Boot verbindet sich über `localhost:5432`. Nach dem Job wird der Container automatisch entfernt — keine Datenbankrückstände.

### Deployment-Preview: Vollständige Testinstanz pro Pull Request

Für eine vollständige Test-Instanz (Frontend + Backend + Datenbank) auf einem Self-hosted Runner:

```

.github/workflows/deploy-preview.yml
name: PR Deployment Preview

```

```

on:
 pull_request:
 branches: [main]
 types: [opened, synchronize, closed]

jobs:
 deploy-preview:
 name: 🚀 Deploy PR Preview
 runs-on: self-hosted
 if: github.event.action != 'closed'

 steps:
 - uses: actions/checkout@v4

 - name: 🐳 Docker-Image bauen
 run: docker build -t schulbibliothek:pr-${{
github.event.pull_request.number }} .

 - name: 🚀 Test-Instanz starten
 run: |
 PORT=$((8080 + ${github.event.pull_request.number}))
 docker compose -f docker-compose.test.yml up -d \
 --project-name "pr-${github.event.pull_request.number}"
 echo "Test-Instanz verfügbar unter: http://schulserver:$PORT"

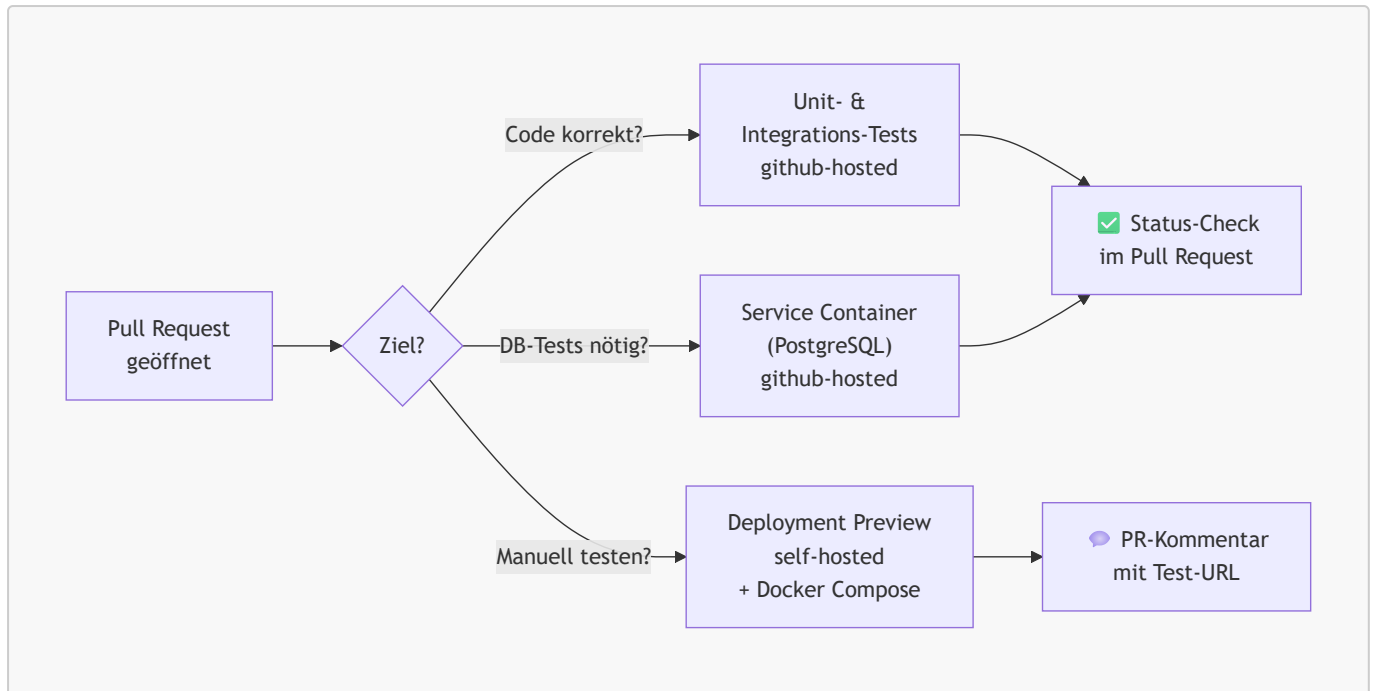
 - name: 💬 PR-Kommentar mit URL erstellen
 uses: actions/github-script@v7
 with:
 script: |
 const port = 8080 + context.issue.number;
 github.rest.issues.createComment({
 issue_number: context.issue.number,
 owner: context.repo.owner,
 repo: context.repo.repo,
 body: `## 🚀 Deployment Preview bereit!\n\nTest-Instanz:
http://schulserver:${port}\n\nWird beim Schließen des PRs automatisch
entfernt.`
 });

 cleanup-preview:
 name: 🧹 Preview aufräumen
 runs-on: self-hosted
 if: github.event.action == 'closed'

 steps:
 - name: 🐳 Docker-Container entfernen
 run: |
 docker compose \
 --project-name "pr-${github.event.pull_request.number}" \
 down --volumes --rmi local

```

**Welcher Ansatz wann?**



Szenario	Ansatz	Runner	Aufwand
<b>Unit-Tests &amp; Build</b>	Standard Workflow	GitHub-hosted	Gering — sofort nutzbar
<b>Integrations-Tests mit DB</b>	Service-Container	GitHub-hosted	Mittel — Docker-Kenntnisse nötig
<b>Vollständige Test-Instanz</b>	Deployment Preview	Self-hosted + Docker	Hoch — eigene Infrastruktur nötig
<b>Plattformspezifische Tests</b>	Self-hosted Runner	Self-hosted	Mittel — einmalige Installation



**Empfehlung für die Schulbibliothek:** Starten Sie mit **GitHub-hosted Runnern** für Unit- und Integrations-Tests (Service Container für PostgreSQL). Ein Self-hosted Runner mit Deployment Preview ist ein spannendes Erweiterungsprojekt für fortgeschrittene Gruppen.

## Quellen

- [Continuous integration - GitHub Docs](#)
- [About service containers - GitHub Docs](#)
- [Adding self-hosted runners - GitHub Docs](#)
- [GitHub Actions: Using Docker service containers](#)